

Chapter 2. Basic Coding in C#

All programming languages have to provide certain capabilities. It must be possible to express the calculations and operations that our code should perform. Programs need to be able to make decisions based on their input. Sometimes we will need to perform tasks repeatedly. These fundamental features are the very stuff of programming, and this chapter will show how these things work in C#.

Depending on your background, some of this chapter's content may seem very familiar. C# is said to be from the “C family” of languages. C is a hugely influential programming language, and numerous languages have borrowed much of its syntax. There are direct descendants, such as C++ and Objective-C. There are also more distantly related languages, including Java, JavaScript, and C# itself, that have no compatibility with C but that still copy many aspects of its syntax. If you are familiar with any of these languages, you will recognize many of the language features we are about to explore.

We saw the basic elements of a program in [Chapter 1](#). In this chapter, we will be looking just at code inside methods. As you've seen, C# requires a certain amount of structure: code is made up of statements that live inside a method, which belongs to a type, which is typically inside a namespace, all inside a file that is part of a project, typically contained by a solution. In the special case of a program's entry point, the containing method and type might be hidden thanks to C#'s clutter-reducing *top-level statements* feature, but they're visible in most files. For clarity, most of the examples in this chapter will show the code of interest in isolation, as in [Example 2-1](#).

Example 2-1. The code and nothing but the code

```
Console.WriteLine("Hello, World!");
```

And although the compiler will accept that shorter example as the entirety of the program, any program larger than a single file (i.e., almost any useful program) will need to include the other elements explicitly. So unless I say otherwise, this kind of extract is shorthand for showing the code in context inside a suitably structured file. Examples such as [Example 2-1](#) are equivalent to something more like [Example 2-2](#).

Example 2-2. The whole code

```
using System;

public class MyType
{
    public static void SomeMethod()
    {
        Console.WriteLine("Hello, World!");
    }
}
```

Although I'll be introducing fundamental elements of the language in this section, this book is for people who are already familiar with at least one programming language, so I'll be relatively brief with the most ordinary features of the language and will go into more detail on those aspects that are particular to C#.

Local Variables

The inevitable “Hello, World!” example is missing a vital element: it doesn't really deal with information. Useful programs normally fetch, process, and produce data, so the ability to define and identify it is one of the most important features of a language. Like most languages, C# lets you define *local variables*, which are named elements inside a method that each hold a piece of information.

NOTE

In the C# specification, the term *variable* can refer to local variables but also to fields in objects and array elements. This section is concerned entirely with local variables, but it gets tiring to keep reading the *local* prefix. So, from now on in this section, *variable* means a local variable.

C# is a *statically typed* language, which is to say that any element of code that represents or produces information, such as a variable or an expression, has a data type determined at compile time. This is different than *dynamically typed* languages, such as JavaScript, in which types are determined at runtime.

The easiest way to see C#'s static typing in action is with simple variable declarations, such as the ones in [Example 2-3](#). Each of these starts with the data type—the first two variables are of type `string`, followed by two `int` variables. These types represent text strings and 32-bit signed integers, respectively.

Example 2-3. Variable declarations

```
string part1 = "the ultimate question";
string part2 = "of something";
int theAnswer = 42;
int andAnotherThing;
```

The data type is followed immediately by the variable's name. The name must begin with either a letter or an underscore, which can be followed by any combination of letters, decimal digits, and underscores. (At least, those are the options if you stick to ASCII. C# supports Unicode, so if you save your file in UTF-8 or UTF-16 format, anything after the first character in an identifier can be any of the characters described in the “Identifier and Pattern Syntax” annex of the Unicode specification. This includes various accents and diacritics, and also numerous punctuation marks, but only those intended for use *within* words—characters that Unicode identifies as being intended for *separating* words cannot be used.) These same rules determine what constitutes a legal identifier for any user-defined entity in C#, such as a class or a method.

[Example 2-3](#) shows that there are a couple of forms of variable declarations. The first three variables include an *initializer*, which provides the variable's initial value, but as the final variable shows, this is optional. That's because you can assign new values into variables at any point. [Example 2-4](#) continues on from [Example 2-3](#) and shows that you can set a variable regardless of whether it had an initial value.

Example 2-4. Assigning values to previously declared variables

```
part2 = " of life, the universe, and everything";  
andAnotherThing = 123;
```

Because variables have a static type, the compiler will reject attempts to assign the wrong kind of data. So if we were to follow on from [Example 2-3](#) with the code in [Example 2-5](#), the compiler would complain. It knows that `theAnswer` is a variable of type `int`, which is a numeric type, so it will report an error if we attempt to assign a text string into it.

Example 2-5. An error: the wrong type

```
theAnswer = "The compiler will reject this";
```

You'd be allowed to do this in dynamic languages such as JavaScript, because in those languages, a variable doesn't have its own type—all that matters is the type of the value it contains, and that can change as the code runs. It's possible to do something similar in C# by declaring a variable with type `dynamic` or `object` (which I'll describe later in [“Dynamic”](#) and [“Object”](#)). However, the most common practice in C# is for variables to have a more specific type.

NOTE

The static type doesn't always provide a complete picture, thanks to inheritance. I'll be discussing this in [Chapter 6](#), but for now, it's enough to know that some types are open to extension through inheritance, and if a variable uses such a type, then it's possible for it to refer to some object of a type derived from the variable's static type. Interfaces, described in [Chapter 3](#), provide a similar kind of flexibility. However, the static type always determines what operations you are allowed to perform on the variable. If you want to use additional members specific to some derived type, you won't be able to do so through a variable of the base type.

You don't have to state the variable type explicitly. You can let the compiler work it out for you by using the keyword `var` in place of the data type. [Example 2-6](#) shows the first three variable declarations from [Example 2-3](#) but using `var` instead of explicit data types.

Example 2-6. Implicit variable types with the `var` keyword

```
var part1 = "the ultimate question";  
var part2 = "of something";  
var theAnswer = 42;
```

This code often misleads people who know some JavaScript, because that also has a `var` keyword that you can use in a similar-looking way. But `var` does not work the same way in C# as in JavaScript: these variables are still all statically typed. All that's changed is that we haven't said what the type is—we're letting the compiler deduce it for us. It looks at the initializers and can see that the first two variables are strings, whereas the third is an integer. (That's why I left out the fourth variable from [Example 2-3](#), `andAnotherThing`. That doesn't have an initializer, so the compiler would have no way of inferring its type. If you try to use the `var` keyword without an initializer, you'll get a compiler error.)

You can demonstrate that variables declared with `var` are statically typed by attempting to assign something of a different type into them. We could repeat the same thing we tried in [Example 2-5](#) but this time with a `var`-style variable. [Example 2-7](#) does this, and it will produce exactly the

same compiler error, because it's the same mistake—we're trying to assign a text string into a variable of an incompatible type. That variable, `theAnswer`, has a type of `int` here, even though we didn't say so explicitly.

Example 2-7. An error: the wrong type (again)

```
var theAnswer = 42;  
theAnswer = "The compiler will reject this";
```

Opinion is divided on how and when to use the `var` keyword, as the sidebar [“To var, or Not to var?”](#) describes.

A variable declared with `var` behaves in exactly the same way as the equivalent explicitly typed declaration, which raises a question: Which should you use? In a sense, it doesn't matter, because they are equivalent. However, if you like your code to be consistent, you'll want to pick one style and stick to it. Not everyone agrees on which is the “best” style.

Some developers see the extra text required for explicit variable types as unproductive “ceremony,” preferring the more succinct `var` keyword. Let the compiler deduce the type for you, instead of doing the work yourself, or so the argument goes. It also reduces visual clutter in the code.

I take a different view, because I spend more time reading code than writing it—debugging, code review, refactoring, and enhancements seem to dominate. Anything that makes those activities easier is worth the frankly minimal time it takes to write the type names explicitly. Code that uses `var` everywhere slows you down, because you have to work out what the type really is in order to understand the code. Although `var` saved you some work when you wrote the code, that gain is quickly wiped out by the additional thought required every time you go back and look at the code. So unless you're the sort of developer who only ever writes new code, leaving others to clean up after you, the only benefit the “`var` everywhere” philosophy really offers is that it can look neater.

You can even use explicit types and still get the compiler to do the work: in Visual Studio, VS Code, or Rider you can write the keystroke-friendly `var`, then press Ctrl-. to open the Quick Actions menu. This offers to replace it with the explicit type for you. (This feature uses the C# compiler's API to discover the variable's type.)

That said, there are some situations in which I will use `var`, such as avoiding writing the name of the type twice. This can happen when you use the `new` operator to create a new instance of some type, as in this example:

```
List<int> numbers1 = new List<int>();  
var numbers2 = new List<int>();
```

```
List<int> numbers3 = new();
```

All three lines have the same effect. The third line works because you can use `new()` without specifying the type as long as the C# compiler can infer the type required. `var n = new();` wouldn't work because that doesn't indicate a type. The type name is right there in all three cases, so the first one doesn't gain anything by stating the type twice, which is why I don't mind the second option, although the third is the most succinct. There are similar examples involving casts and generic methods. As long as the type name appears explicitly in the variable declaration, there is no downside to using `var` to avoid writing the type twice.

I also use `var` where it is necessary. As we will see in later chapters, C# supports *anonymous types*, and as the name suggests, it's not possible to write the name of such a type. In these situations, you may be compelled to use `var`. (In fact, the `var` keyword was introduced to C# only when anonymous types were added.)

You can declare and optionally initialize multiple variables in a single statement. If you want multiple variables of the same type, this may reduce clutter in your code. [Example 2-8](#) declares three variables of the same type in a single declaration and initializes two of them.

Example 2-8. Multiple variables in a single declaration

```
double a, b = 2.5, c = -3;
```

Regardless of how you declare it, a variable holds some piece of information of a particular type, and the compiler prevents us from putting data of an incompatible type into that variable. Variables are useful only because we can refer back to them later in our code. [Example 2-9](#) starts with the variable declarations we saw in earlier examples, then goes on to use the values of those variables to initialize some more variables, and then displays the results.

Example 2-9. Using variables

```
string part1 = "the ultimate question";
string part2 = "of something";
int theAnswer = 42;

part2 = "of life, the universe, and everything";

string questionText = "What is the answer to " + part1 + ", " + part2 + "?";
string answerText = "The answer to " + part1 + ", " +
    part2 + ", is: " + theAnswer;

Console.WriteLine(questionText);
Console.WriteLine(answerText);
```

By the way, this code relies on the fact that C# defines a couple of meanings for the `+` operator when it's used with strings. First, when you “add” two strings together, it concatenates them. Second, when you “add” something other than a string to the end of a string (as the initializer for `answerText` does—it adds `theAnswer`, which is a number), C# generates code that converts the value to a string before appending it. So

[Example 2-9](#) produces this output:

```
What is the answer to the ultimate question, of life, the universe, and everythi
ng?
The answer to the ultimate question, of life, the universe, and everything, is:
42
```

NOTE

In this book, text longer than 80 characters is wrapped across multiple lines to fit the page. If you try these examples, they will look different if your console windows have a different width.

When you use a variable, its value is whatever you last assigned to it. If you attempt to use a variable before you have assigned a value, as [Example 2-10](#) does, the C# compiler will report an error.

Example 2-10. Error: using an unassigned variable

```
int willNotWork;  
Console.WriteLine(willNotWork);
```

Compiling that produces this error for the second line:

```
error CS0165: Use of unassigned local variable 'willNotWork'
```

The compiler uses a slightly pessimistic system (which it calls the *definite assignment* rules) for determining whether a variable has a value yet. It's not possible to create an algorithm that can determine such things for certain in every possible situation.¹ Since the compiler has to err on the side of caution, there are some situations in which the variable will have a value by the time the offending code runs, and yet the compiler still complains. The solution is to write an initializer so that the variable always contains something, perhaps using `0` for numeric values and `false` for Boolean variables. In [Chapter 3](#), I'll introduce reference types, and as the name suggests, a variable of such a type can hold a reference to an instance of the type. If you need to initialize such a variable before you've got something for it to refer to, you can use the keyword `null`, a special value signifying a reference to nothing. Alternatively, you can initialize a variable of any type with the keyword `default`, which denotes a value of zero, `false`, or `null`.

The definite assignment rules determine the parts of your code in which the compiler considers a variable to contain a valid value and will therefore let you read from it. Writing into a variable is less restricted, but as you might expect, any given variable is accessible only from certain parts of the code. Let's look at the rules that govern this.

Scope

A variable's *scope* is the range of code in which you can refer to that variable by its name. Variables are not the only things with scope. Methods, properties, types, and, in fact, anything with a name all have scope. These require broadening the definition of scope: it's the parts of your code

where you can refer to the entity by its name without needing additional qualification. When I write `Console.WriteLine`, I am referring to the method by its name (`WriteLine`), but I need to qualify it with a class name (`Console`), because the method is not in scope. But with a local variable, scope is absolute: either it's accessible without qualification, or it's not accessible at all.

Broadly speaking, a variable's scope starts at its declaration and finishes at the end of its containing *block*. (Some statements, such as loops, complicate this by putting variable declarations ahead of the block in which they are in scope.) A block is a region of code delimited by a pair of braces (`{}`). A method body is a block, so a variable defined in one method is not visible in a separate method, because it is out of scope. If you attempt to compile [Example 2-11](#), you'll get an error complaining that `The name 'thisWillNotWork' does not exist in the current context.`

Example 2-11. Error: out of scope

```
static void SomeMethod()
{
    int thisWillNotWork = 42;
}

static void AnUncompilableMethod()
{
    Console.WriteLine(thisWillNotWork);
}
```

Methods often contain nested blocks, particularly when you work with the loop and flow control constructs we'll be looking at later in this chapter. At the point where a nested block starts, everything that is in scope in the outer block continues to be in scope inside that nested block.

[Example 2-12](#) declares a variable called `someValue` and then introduces a nested block as part of an `if` statement. The code inside this block is able to access that variable declared in the containing block.

Example 2-12. Variable declared outside block, used within block

```
int someValue = GetValue();  
if (someValue > 100)  
{  
    Console.WriteLine(someValue);  
}
```

The converse is not true. If you declare a variable in a nested block, its scope does not extend outside of that block. So [Example 2-13](#) will fail to compile, because the `willNotWork` variable is only in scope within the nested block. The final line of code will produce a compiler error because it tries to use that variable outside of that block.

Example 2-13. Error: trying to use a variable not in scope

```
int someValue = GetValue();  
if (someValue > 100)  
{  
    int willNotWork = someValue - 100;  
}  
Console.WriteLine(willNotWork);
```

This might seem fairly straightforward, but things get a bit more complex when it comes to potential naming collisions. C# sometimes catches people by surprise here.

Variable Name Ambiguity

Consider the code in [Example 2-14](#). This declares a variable called `anotherValue` inside a nested block. As you know, that variable is only in scope to the end of that nested block. After that block ends, we try to declare another variable with the same name.

Example 2-14. Error: surprising name collision

```
int someValue = GetValue();  
if (someValue > 100)
```

```
{
    int anotherValue = someValue - 100; // Compiler error
    Console.WriteLine(anotherValue);
}

int anotherValue = 123;
```

This causes a compiler error on the first of the lines to declare `anotherValue`:

```
error CS0136: A local or parameter named 'anotherValue' cannot be declared in
this scope because that name is used in an enclosing local scope to define a
local or parameter
```

This seems odd. At the final line, the supposedly conflicting earlier declaration is not in scope, because we're outside of the nested block in which it was declared. Furthermore, the second declaration is not in scope within that nested block, because the declaration comes after the block. The scopes do not overlap, but despite this, we're having problems with C#'s rules for avoiding name conflicts. To see why this example fails, we first need to look at a less surprising example.

C# tries to prevent ambiguity by disallowing code where one name might refer to more than one thing. [Example 2-15](#) shows the sort of problem it aims to avoid. Here we've got a variable called `errorCount`, and the code starts to modify this as it progresses,² but partway through, it introduces a new variable in a nested block, also called `errorCount`. It is possible to imagine a language that allowed this—you could have a rule that says that when multiple items of the same name are in scope, you just pick the one whose declaration happened last.

Example 2-15. Error: hiding a variable

```
int errorCount = 0;
if (problem1)
{
    errorCount += 1;

    if (problem2)
```

```

    {
        errorCount += 1;
    }

    // Imagine that in a real program there was a big
    // chunk of code here before the following lines.

    int errorCount = GetErrors(); // Compiler error
    if (problem3)
    {
        errorCount += 1;
    }
}

```

C# chooses not to allow this, because code that did this would be easy to misunderstand. This is an artificially short method because it's a contrived example in a book, making it easy to see the duplicate names, but if the code were a bit longer, it would be very easy to miss the nested variable declaration. Then, we might not realize that `errorCount` refers to something different at the end of the method than it did earlier on. C# simply disallows this to avoid misunderstanding.

But why does [Example 2-14](#) fail? The scopes of the two variables don't overlap. Well, it turns out that the rule that outlaws [Example 2-15](#) is not based on scopes. It is based on a subtly different concept called a *declaration space*. A declaration space is a region of code in which a single name must not refer to two different entities. Each method introduces a declaration space for variables. Nested blocks also introduce declaration spaces, and it is illegal for a nested declaration space to declare a variable with the same name as one in its parent's declaration space. And that's the rule we've contravened here—the outermost declaration space in [Example 2-15](#) contains a variable named `errorCount`, and a nested block's declaration space tries to introduce another variable of the same name.

If that all seems a bit dry or arbitrary, it may be helpful to know *why* there's a whole separate set of rules for name collisions instead of basing it on scopes. The intent of the declaration space rules is that it mostly shouldn't matter where you put the declaration. If you were to move all of the variable declarations in a block to the start of that block—and some

organizations have coding standards that mandate this sort of layout—the idea of these rules is that this shouldn’t change what the code means. This wouldn’t be possible if [Example 2-15](#) were legal. And this explains why [Example 2-14](#) is illegal. Although the scopes don’t overlap, they would if you moved all variable declarations to the tops of their containing blocks.

Local Variable Instances

Variables are features of the source code, so each particular variable has a distinct identity: it is declared in exactly one place in the source code and goes out of scope at exactly one well-defined place. However, that doesn’t mean that it corresponds to a single storage location in memory. It is possible for multiple invocations of a single method to be in progress simultaneously, through recursion, multithreading, or asynchronous execution.

Each time a method runs, it gets a distinct set of storage locations to hold the local variables’ values. This enables multiple threads to execute the same method simultaneously without problems, because each has its own set of local variables. Likewise, in recursive code, each nested call gets its own set of locals that will not interfere with any of its callers. The same goes for multiple concurrent invocations of an asynchronous method. To be strictly accurate, each execution of a particular *scope* gets its own set of variables. This distinction matters when you use anonymous functions, described in [Chapter 9](#). As an optimization, C# reuses storage locations when it can, so it will only allocate new memory for each scope’s execution when it really has to. (For example, it won’t allocate new memory for variables declared in the body of a loop for each iteration unless you put it into a situation where it has no choice.) But the effect is as though it allocated new space each time.

Be aware that the C# compiler does not make any particular guarantee about where variables live (except for some special cases, as we’ll see in [Chapter 18](#)). They might well live on the stack, but sometimes they don’t. When we look at anonymous functions in later chapters, you’ll see that variables sometimes need to outlive the method that declares them, be-

cause they remain in scope for nested methods that will run as callbacks after the containing method has returned.

By the way, before we move on, be aware that just as variables are not the only things to have scope, they are also not the only things to which declaration space rules apply. Other language features that we'll be looking at later, including classes, methods, and properties, also have scoping and name uniqueness rules.

Statements and Expressions

Variables give us somewhere to put the information that our code works with, but to do anything with those variables, we will need to write some code. This will mean writing *statements* and *expressions*.

Statements

When we write a C# method, we are writing a sequence of statements. Informally, the statements in a method describe the actions we want the method to perform. Each line in [Example 2-16](#) is a statement. It might be tempting to think of a statement as an instruction to do one thing (such as initializing a variable or invoking a method). Or you might take a more lexical view, where anything ending in a semicolon is a statement. (And it's the semicolons that are significant here, not the line breaks, by the way. I could have written this as one long line of code and it would have exactly the same meaning.) However, both descriptions are simplistic, even though they happen to be true for this particular example.

Example 2-16. Some statements

```
int a = 19;  
int b = 23;  
int c;  
c = a + b;  
Console.WriteLine(c);
```

C# recognizes many different kinds of statements. In [Example 2-16](#), the first three lines are *declaration statements*, statements that declare and optionally initialize a variable. The fourth and fifth lines are *expression statements*. But some statements have more structure than the ones in this example.

When you write a loop, that's an *iteration statement*. When you use the `if` or `switch` mechanisms described later in this chapter to choose between various possible actions, those are *selection statements*. In fact, the C# specification distinguishes between 13 categories of statements. Most fit broadly into the scheme of describing either what the code should do next or, for features such as loops or conditional statements, describing how it should decide what to do next. Statements of the second kind usually contain one or more embedded statements describing the action to perform in a loop, or the action to perform when an `if` statement's condition is met.

There's one special case, though. A block is a kind of statement. This makes statements such as loops more useful than they would otherwise be, because a loop iterates over just a single embedded statement. That statement can be a block, and since a block itself is a sequence of statements (delimited by braces), this enables loops to contain more than one statement.

This illustrates why the two simplistic points of view stated earlier—"statements are actions" and "statements are things that end in semicolons"—are wrong. Compare [Example 2-16](#) with [2-17](#). Both do the same thing, because the various actions we've said we want to perform remain exactly the same, and both contain five semicolons. However, [Example 2-17](#) contains one extra statement. The first two statements are the same, but they are followed by a third statement, a block, which contains the final three statements from [Example 2-16](#). The extra statement, the block, doesn't end in a semicolon, nor does it perform any action. In this particular example, it's pointless, but it can sometimes be useful to introduce a nested block like this to avoid name ambiguity errors. So statements can be structural, rather than causing anything to happen at runtime.

Example 2-17. A block

```
int a = 19;
int b = 23;
{
    int c;
    c = a + b;
    Console.WriteLine(c);
}
```

While your code will use a mixture of statement types, it will inevitably end up containing at least a few expression statements. An expression statement is a statement that consists of a suitable expression, followed by a semicolon. What’s a suitable expression? What’s an expression, for that matter? I’d better answer that second question before coming back to what constitutes a valid expression for a statement.

Expressions

The official definition of a C# *expression* is rather dry: “a sequence of operators and operands.” Admittedly, language specifications tend to be like that, but in addition to this sort of formal prose, the C# specification contains some very readable informal explanations of the more formally expressed ideas. (For example, it describes statements as the means by which “the actions of a program are expressed” before going on to pin that down with less approachable but more technically precise language.) The quote at the start of this paragraph is from the formal definition of an expression, so we might hope that the informal explanation in the introduction will be more helpful. No such luck: it says that expressions “are constructed from operands and operators.” That’s certainly less precise than the other definition, but it’s no easier to understand. The problem is that there are several kinds of expressions, and they do different jobs, so there isn’t a single, general, informal description.

It’s tempting to describe an expression as some code that produces a value. That’s not true for all expressions, but the majority of expressions you’ll write will fit this description, so I’ll focus on this for now, and I’ll come to the exceptions later.

The simplest expressions are *literals*, where we just write the value we want, such as `"Hello, World!"` or `42`. You can also use the name of a variable as an expression. Expressions can involve operators, which describe calculations or other computations to be performed. Operators have some fixed number of inputs, called *operands*. Some take a single operand. For example, you can negate a number by putting a minus sign in front of it. Some take two: the `+` operator lets you form an expression that adds together the results of the two operands on either side of the `+` symbol.

NOTE

Some symbols have different roles depending on the context. The minus sign is not just used for negation. It acts as a two-operand subtraction operator if it appears between two expressions.

In general, operands are also expressions. So, when we write `2 + 2`, that's an expression that contains two more expressions—the pair of `"2"` literals on either side of the `+` symbol. This means that we can write arbitrarily complicated expressions by nesting expressions within expressions within expressions. [Example 2-18](#) exploits this to evaluate the quadratic formula (the standard way to solve quadratic equations).

Example 2-18. Expressions within expressions

```
double a = 1, b = 2.5, c = -3;  
double x = (-b + Math.Sqrt(b * b - 4 * a * c)) / (2 * a);  
Console.WriteLine(x);
```

Look at the declaration statement on the second line. The overall structure of its initializer expression is a division operation. But that division operator's two operands are also expressions. Its lefthand operand is a *parenthesized expression*, which tells the compiler that I want that whole expression `(-b + Math.Sqrt(b * b - 4 * a * c))` to be the first operand of the division. This subexpression contains an addition, whose lefthand operand is a negation expression whose single operand is the variable `b`. The addition's righthand side takes the square root of another,

more complex expression. And the division's righthand operand is another parenthesized expression, containing a multiplication. [Figure 2-1](#) illustrates the full structure of the expression.

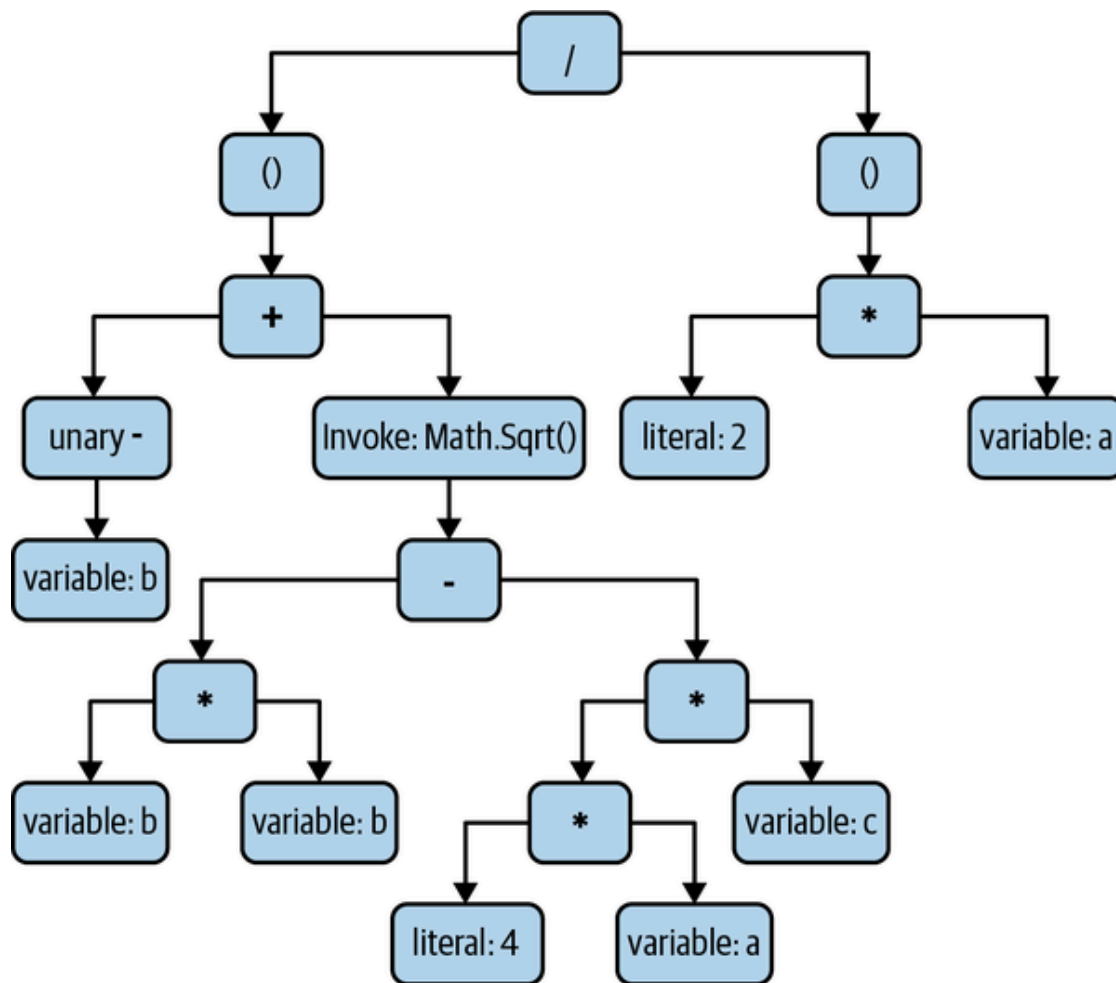


Figure 2-1. The structure of an expression

One important detail of this example is that method invocations are a kind of expression. The `Math.Sqrt` method used in [Example 2-18](#) is a .NET runtime library function that calculates the square root of its input and returns the result. What's perhaps more surprising is that invocations of methods that don't return a value, such as `Console.WriteLine`, are also, technically, expressions. And there are a few other constructs that don't produce values but are still considered to be expressions, including a reference to a type (such as the `Console` in `Console.WriteLine`) or to a namespace. These sorts of constructs take advantage of a set of common rules (such as scoping, how to resolve what a name refers to, etc.) by virtue of being expressions. However, all the non-value-producing expressions can be used only in certain specific circumstances. (You can't use one as an operand in another expression, for

example.) So although it's not technically correct to define an expression as a piece of code that produces a value, the ones that do are the ones we use when describing the calculations we want our code to perform.

We can now return to the question: what can we put in an expression statement? Roughly speaking, the expression has to have some effect; it cannot just calculate a value. So although `2 + 2` is a valid expression, you'll get an error if you try to turn it into an expression statement by sticking a semicolon on the end. That expression calculates something but doesn't do anything with the result. To be more precise, you can use the following kinds of expressions as statements: method invocation, assignment, increment, decrement, and new object creation. We'll be looking at increment and decrement later in this chapter, and we'll be looking at objects in later chapters, so that leaves invocation and assignment.

When we write a method invocation as an expression statement, there may be nested expressions of other kinds (such as the method's arguments), but the whole thing must be a method call. [Example 2-19](#) shows some valid examples. Notice that the C# compiler doesn't check whether the method call really has any lasting effect—the `Math.Sqrt` function is a pure function, in the sense that it does nothing other than returning a value determined entirely by its inputs. So invoking it and then doing nothing with the result doesn't really do anything at all—it's no more of an action than the expression `2 + 2`. But as far as the C# compiler is concerned, any method call is allowed as an expression statement.

Example 2-19. Method invocation expressions as statements

```
Console.WriteLine("Hello, World!");  
Console.WriteLine(12 + 30);  
Console.ReadKey();  
Math.Sqrt(4);
```

NOTE

If you run this example in VS Code, the call to `ReadKey` might fail because of how the debugger redirects input and output by default. The [documentation explains](#) how to avoid this problem when debugging programs that need to read console input.

It seems inconsistent that C# forbids us from using an addition expression as a statement while allowing `Math.Sqrt`. Both perform a calculation that produces a result, so it makes no sense to use either in this way. Wouldn't it be more consistent if C# allowed only calls to methods that return nothing to be used for expression statements? That would rule out the final line of [Example 2-19](#), which would seem like a good idea because that code does nothing useful. It would also be consistent with the fact that `2 + 2` also cannot form an expression statement. Unfortunately, sometimes you want to ignore the return value. [Example 2-19](#) calls `Console.ReadKey()`, which waits for a keypress and returns a value indicating which key was pressed. If my program's behavior depends on which particular key the user pressed, I'll need to inspect the method's return value, but if I just want to wait for any key at all, it's OK to ignore the return value. If C# didn't allow methods with return values to be used as expression statements, I wouldn't be able to do this. The compiler has no way to distinguish between methods that make for pointless statements because they have no side effects (such as `Math.Sqrt`) and those that might be good candidates (such as `Console.ReadKey`), so it allows any method.

For an expression to be a valid expression statement, it is not enough merely to contain a method invocation. [Example 2-20](#) shows some expressions that call methods and then go on to use those as part of addition expressions. Although these are valid expressions, they're not valid expression statements, so these will cause compiler errors. What matters is the outermost expression. In both lines here, that's an addition expression, which is why these are not allowed.

Example 2-20. Errors: some expressions that don't work as

statements

```
Console.ReadKey().KeyChar + "!";  
Math.Sqrt(4) + 1;
```

Earlier I said that one kind of expression we're allowed to use as a statement is an assignment. It's not obvious that assignments should be expressions, but they are, and they do produce a value: the result of an assignment expression is the value being assigned to the variable. This means it's legal to write code like that in [Example 2-21](#). The second line here uses an assignment expression as an argument for a method invocation, which shows the value of that expression. The first two `WriteLine` calls both display `123`.

Example 2-21. Assignments are expressions

```
int number;  
Console.WriteLine(number = 123);  
Console.WriteLine(number);  
  
int x, y;  
x = y = 0;  
Console.WriteLine(x);  
Console.WriteLine(y);
```

The second part of this example assigns one value into two variables in a single step by exploiting the fact that assignments are expressions—it assigns the value of the `y = 0` expression (which evaluates to `0`) into `x`.

This shows that evaluating an expression can do more than just produce a value. Some expressions have side effects. We've just seen that an assignment is an expression, and of course it has the effect of changing what's in a variable. Method calls are expressions too, and although you can write pure functions that do nothing besides calculating their result from their input, like `Math.Sqrt`, many methods do something with lasting effects, such as writing data to the screen, updating a database, or triggering the demolition of a building. This means that we might care about the order in which the operands of an expression get evaluated.

An expression's structure imposes some constraints on the order in which operators do their work. For example, I can use parentheses to enforce ordering. The expression `10 + (8 / 2)` has the value 14, while the expression `(10 + 8) / 2` has the value 9, even though both have exactly the same literal operands and arithmetic operators. The parentheses here determine whether the division is performed before or after the addition.³

However, while the structure of an expression imposes some ordering constraints, it still leaves some latitude: although both the operands of an addition need to be evaluated before they can be added, the addition operator doesn't care which operand we evaluate first. But if the operands are expressions with side effects, the order could be important. For these simple expressions, it doesn't matter because I've used literals, so we can't really tell when they get evaluated. But what about an expression in which operands call some method? [Example 2-22](#) contains code of this kind.

Example 2-22. Operand evaluation order

```
static int X(string label, int i)
{
    Console.Write(label);
    return i;
}

Console.WriteLine(X("a", 1) + X("b", 1) + X("c", 1) + X("d", 1));
```

This defines a method, `X`, which takes two arguments. It displays the first and just returns the second. I've then used this a few times in an expression so that we can see exactly when the operands that call `X` are evaluated. Some languages choose not to define this order, making the behavior of such a program unpredictable, but C# does specify an order here. The rule is that within any expression, the operands are evaluated in the order in which they occur in the source. So, when the `Console.WriteLine` in [Example 2-22](#) runs, it makes multiple calls to `X`, which calls `Console.Write` each time, so we see this output: `abcd4`.

However, this glosses over an important subtlety: What do we mean by the order of expressions when nesting occurs? The entire argument to that `Console.WriteLine` is one big add expression, where the first operand is `X("a", 1)`, and the second is another add expression, which in turn has a first operand of `X("b", 1)` and a second operand, which is yet another add expression, whose operands are `X("c", 1)` and `X("d", 1)`. Taking the first of those add expressions, which constitutes the entire argument to `Console.WriteLine`, does it even make sense to ask whether it comes before or after its first operand? Lexically, the outermost add expression starts at exactly the same point that its first operand starts and ends at the point where its second operand ends (which also happens to be at the exact same point that the final `X("d", 1)` ends). In this particular case, it doesn't really matter because the only observable effect of the order of evaluation is the output the `X` method produces when invoked. None of the expressions that invoke `X` are nested within one another, so we can meaningfully say what order those expressions are in, and the output we see matches that order. However, in some cases, such as [Example 2-23](#), the overlapping of nested expressions can have a visible impact.

Example 2-23. Operand evaluation order with nested expressions

```
Console.WriteLine(  
    X("a", 1) +  
    X("b", (X("c", 1) + X("d", 1) + X("e", 1))) +  
    X("f", 1));
```

Here, `Console.WriteLine`'s argument adds the results of three calls to `X`; however, the second of those calls to `X` (first argument `"b"`) takes as its second argument an expression that adds the results of three more calls to `X` (with arguments of `"c"`, `"d"`, and `"e"`). With the final call to `X` (passing `"f"`), we have a total of six expressions invoking `X` in that statement. C#'s rule of evaluating expressions in the order in which they appear applies as always, but because there is overlap, the results are initially surprising. Although the letters appear in the source in alphabetical order, the output is `"acdeb5"`. If you're wondering how on earth that can be consistent with expressions being evaluated in order, consider that this code starts the evaluation of each expression in the order in

which the expressions start, and finishes the evaluation in the order in which the expressions finish, but that those are two different orderings. In particular, the expression that invokes `x` with `"b"` begins its evaluation before those that invoke it with `"c"`, `"d"`, and `"e"`, but it finishes its evaluation *after* them. And it's that *after* ordering that we see in the output. If you find each closing parenthesis that corresponds to a call to `x` in this example, you'll find that the order of calls exactly matches what's displayed.

Comments and Whitespace

Most programming languages allow source files to contain text that is ignored by the compiler, and C# is no exception. As with most C-family languages, it supports two styles of *comments* for this purpose. There are *single-line comments*, as shown in [Example 2-24](#), in which you write two `/` characters in a row, and everything from there to the end of the line will be ignored by the compiler.

Example 2-24. Single-line comments

```
Console.WriteLine("Say");           // This text will be ignored, but the code on
Console.WriteLine("Anything");      // the left is still compiled as usual.
```

C# also supports *delimited comments*. You start a comment of this kind with `/*`, and the compiler will ignore everything that follows until it encounters the first `*/` character sequence. This can be useful if you don't want the comment to go all the way to the end of the line, as the first line of [Example 2-25](#) illustrates. This example also shows that delimited comments can span multiple lines.

Example 2-25. Delimited comments

```
Console.WriteLine(/* Has side effects */ GetLog());

/* Some developers like to use delimited comments for big blocks of text,
 * where they need to explain something particularly complex or odd in the
 * code. The column of asterisks on the left is for decoration - asterisks
```

```
* are necessary only at the start and end of the comment.  
*/
```

There's a minor snag you can run into with delimited comments; it can happen even when the comment is within a single line, but it more often occurs with multiline comments. [Example 2-26](#) shows the problem with a comment that begins in the middle of the first line and ends at the end of the fourth.

Example 2-26. Multiline comments

```
Console.WriteLine("This will run");    /* This comment includes not just the  
Console.WriteLine("This won't");        * text on the right but also the text  
Console.WriteLine("Nor will this");     /* on the left except the first and last  
Console.WriteLine("Nor this");          * lines. */  
Console.WriteLine("This will also run");
```

Notice that the `/*` character sequence appears twice in this example. When this sequence appears in the middle of a comment, it does nothing special—comments don't nest. Even though we've seen two `/*` sequences, the first `*/` is enough to end the comment. This is occasionally frustrating, but it's the norm for C-family languages.

It's sometimes useful to take a chunk of code out of action temporarily, in a way that's easy to put back. Turning the code into a comment is a common way to do this, and although a delimited comment might seem like the obvious thing to use, it becomes awkward if the region you commented out happens to include another delimited comment. Since there's no support for nesting, you would need to add a `/*` after the inner comment's closing `*/` to ensure that you've commented out the whole range. So it is common to use single-line comments for this purpose. (You can also use the `#if` directive described in the next section.)

NOTE

Visual Studio and VS Code can comment out regions of code for you. If you select several lines of text and type Ctrl-K followed immediately by Ctrl-C, it will add `//` to the start of every line in the selection. And you can uncomment a region with Ctrl-K, Ctrl-U. (With Visual Studio, if you chose something other than C# as your preferred language when you installed it, these actions may be bound to different key sequences, but they are also available on the Edit→Advanced menu, as well as on the Text Editor toolbar, one of the standard toolbars that Visual Studio shows by default.)

Speaking of ignored text, C# ignores extra whitespace for the most part. Not all whitespace is insignificant, because you need at least some space to separate tokens that consist entirely of alphanumeric symbols. For example, you can't write `staticvoid` as the start of a method declaration—you'd need at least one space (or tab, newline, or other space-like character) between `static` and `void`. But with nonalphanumeric tokens, spaces are optional, and in most cases, a single space is equivalent to any amount of whitespace and new lines. This means that the three statements in [Example 2-27](#) are all equivalent.

Example 2-27. Insignificant whitespace

```
Console.WriteLine("Testing");
Console . WriteLine(  "Testing");
Console.
    WriteLine ("Testing" )
;
```

There are a couple of cases where C# is more sensitive to whitespace. Inside a string literal, space is significant, because whatever spaces you write will be present in the string value. Also, while C# mostly doesn't care whether you put each element on its own line, or put all your code in one massive line, or (as seems more likely) something in between, there is an exception: preprocessing directives are required to appear on their own lines.

Preprocessing Directives

If you're familiar with the C language or its direct descendants, you may have been wondering if C# has a preprocessor. It doesn't have a separate preprocessing stage, and it does not offer macros. However, it does have a handful of directives similar to those offered by the C preprocessor, although it is only a very limited selection. Even though C# doesn't have a full preprocessing stage like C, these are known as preprocessing directives nonetheless.

Compilation Symbols

C# offers a `#define` directive that lets you define a *compilation symbol*. These symbols are commonly used in conjunction with the `#if` directive to compile code in different ways for different situations. For example, you might want some code to be present only in Debug builds, or perhaps you need to use different code on different platforms to achieve a particular effect. Often, you won't use the `#define` directive, though—it's more common to define compilation symbols through the compiler build settings. You can open up the `.csproj` file and define the values you want in a `<DefineConstants>` element of any `<PropertyGroup>`. Alternatively, Visual Studio can do this for you: right-click the project's node in Solution Explorer, select Properties, and in the property page that this opens, go to the Build section. This UI lets you configure different symbol values for each build configuration (which it does by adding attributes such as `Condition="'$(Configuration)|$(Platform)'=='Debug|AnyCPU'"` to the `<PropertyGroup>` containing these settings).

NOTE

The .NET SDK defines certain symbols by default. It supports two configurations, Debug and Release. It defines a `DEBUG` compilation symbol in the Debug configuration, whereas Release will define `RELEASE` instead. It defines a symbol called `TRACE` in both configurations. Certain project types get additional symbols. A library targeting .NET Standard will have `NETSTANDARD` defined, along with a version-specific symbol such as `NETSTANDARD2_0`, for example. Projects that target .NET 8.0 get a `NET8_0` symbol.

Compilation symbols are typically used in conjunction with the `#if`, `#else`, `#elif`, and `#endif` directives (`#elif` is short for *else if*).

[Example 2-28](#) uses some of these directives to ensure that certain lines of code get compiled only in Debug builds. (You can also write `#if false` to prevent sections of code from being compiled at all. This is typically done only as a temporary measure and is an alternative to commenting out that sidesteps some of the lexical pitfalls of attempting to nest comments.)

Example 2-28. Conditional compilation

```
#if DEBUG
    Console.WriteLine("Starting work");
#endif
    DoWork();
#if DEBUG
    Console.WriteLine("Finished work");
#endif
```

C# provides a more subtle mechanism to support this sort of thing, called a *conditional method*. The compiler recognizes an attribute defined by the runtime libraries, called `ConditionalAttribute`, for which it provides special compile-time behavior. You can annotate any method with this attribute. [Example 2-29](#) uses it to indicate that the annotated method should be used only when the `DEBUG` compilation symbol is defined.

Example 2-29. Conditional method

```
[System.Diagnostics.Conditional("DEBUG")]
static void ShowDebugInfo(object o)
{
    Console.WriteLine(o);
}
```

If you write code that calls a method that has been annotated in this way, the C# compiler will omit that call in builds that do not define the relevant symbol. So if you write code that calls this `ShowDebugInfo` method, the compiler strips out all those calls in non-Debug builds. This means

you can get the same effect as [Example 2-28](#) but without cluttering up your code with directives.

The runtime libraries' `Debug` and `Trace` classes in the `System.Diagnostics` namespace use this feature. The `Debug` class offers various methods for generating diagnostic output that are conditional on the `DEBUG` compilation symbol, while the `Trace` class has methods conditional on `TRACE`. If you leave the default settings for a new C# project in place, any diagnostic output produced through the `Trace` class will be available in both Debug and Release builds, but any code that calls a method on the `Debug` class will not get compiled into Release builds.

WARNING

The `Debug` class's `Assert` method is conditional on `DEBUG`, which sometimes catches developers out. `Assert` lets you specify a condition that must be true at runtime, and it throws an exception if the condition is false. There are two things developers new to C# sometimes mistakenly put in a `Debug.Assert`: checks that should in fact occur in all builds, and expressions with side effects that the rest of the code depends on. This leads to bugs, because the compiler will strip this code out in non-Debug builds.

#error and #warning

C# lets you choose to generate compiler errors or warnings with the `#error` and `#warning` directives. These are typically used inside conditional regions, as [Example 2-30](#) shows, although an unconditional `#warning` could be useful as a way to remind yourself that you've not written some particularly important bit of the code yet.

Example 2-30. Generating a compiler error

```
#if NETSTANDARD
    #error .NET Standard is not a supported target for this source file
#endif
```

#line

The `#line` directive is useful in generated code. When the compiler produces an error or a warning, it states where the problem occurred, providing the filename, a line number, and an offset within that line. But if the code in question was generated automatically using some other file as input and if that other file contains the root cause of the problem, it may be more useful to report an error in the input file, rather than the generated file. A `#line` directive can instruct the C# compiler to act as though the error occurred at the line number specified and, optionally, as if the error were in an entirely different file. [Example 2-31](#) shows how to use it. The error after the directive will be reported as though it came from line 123 of a file called *Foo.cs*. You can tell the compiler to revert to reporting warnings and errors without fakery by writing `#line default`.

Example 2-31. The `#line` directive and a deliberate mistake

```
#line 123 "Foo.cs"
    intt x;
```

This directive also affects debugging. When the compiler emits debug information, it takes `#line` directives into account. This means that when stepping through code in the debugger, you'll see the location that `#line` refers to.

There's another use for this directive. Instead of a line number (and optional filename), you can write just `#line hidden`. This affects only the debugger behavior: when single stepping, Visual Studio will run straight through all the code after such a directive without stopping until it encounters a non-`hidden` `#line` directive (typically `#line default`).

#pragma

The `#pragma` directive provides two features: it can be used to disable selected compiler warnings, and it can also be used to override the checksum values the compiler puts into the *.pdb* file it generates containing debug information. Both of these are designed primarily for code-genera-

tion scenarios, although it can occasionally be useful to disable warnings in ordinary code. [Example 2-32](#) shows how to use a `#pragma` to prevent the compiler from issuing the warning that would normally occur if you declare a variable that you do not then go on to use.

Example 2-32. Disabling a compiler warning

```
#pragma warning disable CS0168  
int a;
```

You should generally avoid disabling warnings. This feature is useful in generated code because code generation can often end up creating items that are not always used, and pragmas may offer the only way to get a clean compilation. But when you're writing code by hand, it should usually be possible to avoid normal compiler warnings in the first place.

Having said that, it can be useful to disable specific warnings if you have opted in to additional diagnostics. Some components on NuGet supply *code analyzers*, components that get connected up to the C# compiler API and that are given the opportunity to inspect the code and generate their own diagnostic messages. (This happens at build time, and in Visual Studio, it also happens during editing, providing live diagnostics as you type. They also work live in Visual Studio Code if you install the C# Dev Kit extension.) The .NET SDK also includes built-in analyzers that can check various aspects of your code such as adherence to naming conventions or the presence of common security mistakes. You can configure these at a project level with the `AnalysisMode` setting, but as with compiler warnings, you might want to disable analyzer warnings in specific cases. You can use `#pragma warning` directives to control warnings from code analyzers, not just ones from the C# compiler. Analyzers generally prefix their warning numbers with some letters to enable you to distinguish between them—compiler warnings all start with `CS`, and warnings from the .NET SDK's analyzers start with `CA`, for example.

It's possible that future versions of C# may add other features based on `#pragma`. When the compiler encounters a pragma it does not understand, it generates a warning, not an error, on the grounds that an unrec-

ognized pragma might be valid for some future compiler version or some other vendor's compiler.

#nullable

The `#nullable` directive allows fine-grained control of the nullable annotation context and the nullable warning context. This is part of the *nullable references* feature. [Chapter 3](#) describes the `#nullable` directive in more detail.

#region and #endregion

Finally, we have two preprocessing directives that do nothing. If you write `#region` directives, the only thing the compiler does is ensure that they have corresponding `#endregion` directives. Mismatches cause compiler errors, but the compiler ignores correctly paired `#region` and `#endregion` directives. Regions can be nested.

These directives exist entirely for the benefit of text editors that choose to recognize them. Visual Studio, VS Code, and Rider use them to provide the ability to collapse sections of the code down to a single line on screen. These editors automatically allow certain features to be expanded and collapsed, such as class definitions, methods, and code blocks (a feature they call *outlining*). If you define regions with these two directives, these can also be expanded and collapsed. This allows for outlining at both finer-grained (for example, within a single block) and coarser-grained (for example, multiple related methods) scales than the editor offers automatically.

If you hover the mouse over a collapsed region in Visual Studio, it displays a tool tip showing the region's contents. You can put text after the `#region` token. When IDEs display a collapsed region, they show this text on the single line that remains. Although you're allowed to omit this, it's usually a good idea to include some descriptive text so that people can have a rough idea of what they'll see if they expand it.

Some people like to put the entire contents of a class into various regions, because by collapsing all regions, you can see a file's structure at a glance. It might even all fit on the screen at once, thanks to the regions being reduced to a single line. On the other hand, some people hate collapsed regions, because they present speed bumps on the way to being able to look at the code and can also encourage people to put too much source code into one file.

Fundamental Data Types

.NET defines thousands of types in its runtime libraries, and you can write your own, so C# can work with an unlimited number of data types. However, a handful of types get special handling from the compiler. You saw earlier in [Example 2-9](#) that if you have a string, and you try to add a number to it, the resulting code converts the number to a string and appends that to the first string. In fact, the behavior is more general than that—it's not limited to numbers. The compiled code works by calling the `String.Concat` method, and if you pass to that any nonstring arguments, it will call their `ToString` methods before performing the append. All types offer a `ToString` method, so this means you can append values of any type to a string.

That's handy, but it only works because the C# compiler knows about strings and provides special services for them. (There's a part of the C# specification that defines the unique string handling for the `+` operator.) C# provides various special services not just for strings but also for certain numeric data types, Booleans, a family of types called tuples, and two specific types called `dynamic` and `object`. Most of these are special not just to C# but also to the runtime—almost all of the numeric types get direct support in intermediate language (IL), and the `bool`, `string`, and `object` types are also intrinsically understood by the runtime.

Numeric Types

C# supports integer and floating-point arithmetic. The CLR provides intrinsic support for signed and unsigned integer types of various sizes, as [Table 2-1](#) shows. The most commonly used integer type is `int`, not least

because it is large enough to represent a usefully wide range of values without being too large to work efficiently on all CPUs that support .NET. (Larger data types might not be handled natively by the CPU and can also have undesirable characteristics in multithreaded code: reads and writes are atomic for 32-bit types⁴ but may not be for larger ones.)

Table 2-1. Intrinsic integer types

C# type	CLR name	Signed	Size in bits	Inclusive range
byte	System.Byte	No	8	0 to 255
sbyte	System.SByte	Yes	8	-128 to 127
ushort	System.UInt16	No	16	0 to 65,535
short	System.Int16	Yes	16	-32,768 to 32,767
uint	System.UInt32	No	32	0 to 4,294,967,295
int	System.Int32	Yes	32	-2,147,483,648 to 2,147,483,647
ulong	System.UInt64	No	64	0 to 18,446,744,073,709,551,615
long	System.Int64	Yes	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
nuint	System.UIntPtr	No	Depends	Depends
nint	System.IntPtr	Yes	Depends	Depends

The second column in [Table 2-1](#) shows the name of the type in the CLR. Different languages have different naming conventions, and C# uses

names from its C-family roots for numeric types, but those don't fit with the naming conventions that .NET has for its data types. As far as the runtime is concerned, the names in the second column are the real names—there are various APIs that can report information about types at runtime, and they report these CLR names, not the C# ones. The names are synonymous in C# source code, so you're free to use the runtime names if you want to, but the C# names are a better stylistic fit—keywords in C-family languages are all lowercase. Since the compiler handles these types differently than the rest, it's arguably good to have them stand out.

The `nint` and `nuint` types are unusual in that their size is not always the same. These are the *native-sized integer* types (hence the `n` prefix), and they are intended for low-level code that needs to deal directly with the address of data in memory. This is why they don't have a fixed size—they are 32 bits wide in a 32-bit process and 64 bits in a 64-bit process. (These are very specialized types, normally only used when writing wrappers for non-.NET libraries, and I've included them in this table only for completeness.) These types are aliases for `System.IntPtr` and `System.UIntPtr`, although this was not true before C# 11.0. The .NET runtime didn't used to support arithmetic with `IntPtr` and `UIntPtr`, and the `nint` and `nuint` types were originally introduced as a work-around for this. If you declared variables in C# as `nint` or `nuint`, their runtime types would be `IntPtr` or `UIntPtr`, but you could perform pointer arithmetic on them and the C# compiler would generate the necessary code to make that work. However, .NET 7.0 added arithmetic support for `IntPtr` and `UIntPtr` as part of the *generic math* feature (see [Chapter 4](#)) meaning C# no longer needed to offer `nint` and `nuint` as extended versions of these types to enable arithmetic, so these are now merely aliases like everything else in the table.

WARNING

Not all .NET languages support unsigned numbers, so the .NET runtime libraries tend to avoid them. A runtime that supports multiple languages (such as the CLR) faces a trade-off between offering a type system rich enough to cover most languages' needs and forcing an overcomplicated type system on simple languages. To resolve this, .NET's type system, the CTS, is reasonably comprehensive, but languages don't have to support all of it. .NET also defines the Common Language Specification (CLS), which identifies a relatively small subset of the CTS that all languages should support. Signed integers are in the CLS, but unsigned ones are not. This explains some surprising-looking type choices, such as the `Length` property of an array being `int` (rather than `uint`) despite the fact that it will never return a negative value.

C# also supports floating-point numbers. It has intrinsic support for two types: `float` and `double`, which are 32-bit and 64-bit numbers in the [standard IEEE 754 formats](#), and as the CLR names in [Table 2-2](#) suggest, these correspond to what are commonly called *single-precision* and *double-precision numbers*. Floating-point values do not work in the same way as integers, so this table is a little different than the integer types table. Floating-point numbers store a value and an exponent (similar in concept to scientific notation but working in binary instead of decimal). The Precision column shows how many bits are available for the value part, and then the range is expressed as the smallest nonzero value and the largest value that can be represented. (These can be either positive or negative.)

Table 2-2. Intrinsic floating-point types

C# type	CLR name	Size in bits	Precision	Range (magnitude)
<code>float</code>	<code>System.Single</code>	32	23 bits (~7 decimal digits)	1.5×10^{-45} to 3.4×10^{38}
<code>double</code>	<code>System.Double</code>	64	52 bits (~15 decimal digits)	5.0×10^{-324} to 1.8×10^{308}

C# recognizes another noninteger numeric representation called `decimal` (or `System.Decimal` in the CLR). This is a 128-bit value, so it can

offer greater precision than the other formats, but it is not just a bigger version of `double`. It is designed for calculations that require predictable handling of decimal fractions, something neither `float` nor `double` can offer. If you write code that initializes a variable of type `float` to 0 and then adds 0.1 to it nine times in a row, you might expect to get a value of 0.9, but in fact you'll get approximately 0.9000001. That's because IEEE 754 stores numbers in binary, which cannot represent all decimal fractions. It can handle some, such as the decimal 0.5; written in base 2, that's 0.1. But the decimal 0.1 turns into a recurring number in binary. (Specifically, it's 0.0 followed by the recurring sequence 0011.) This means `float` and `double` can represent only an approximation of the decimal value 0.1, and more generally, only a small subset of decimals can be represented completely accurately. This isn't always instantly obvious, because when floating-point numbers are converted to text, they are rounded to a decimal approximation that can mask the discrepancy. But over multiple calculations, the inaccuracies tend to add up and eventually produce surprising-looking results.

For some kinds of calculations, the lack of exact decimal fidelity doesn't really matter; in simulations or signal processing, for example, some noise and error is expected. But accountants and financial regulators tend to be less forgiving—little discrepancies like this can make it look like money has magically vanished or appeared. We need calculations that involve money to be absolutely accurate, which makes binary floating point a terrible choice for such work. This is why C# offers the `decimal` type, which provides a well-defined level of decimal precision.

NOTE

Most of the integer types can be handled natively by the CPU. (All of them can when running in a 64-bit process.) Likewise, many CPUs can work directly with `float` and `double` representations. However, none has intrinsic support for `decimal`, meaning that even simple operations, such as addition, require multiple CPU instructions. This means that arithmetic is significantly slower with `decimal` than with the other numeric types shown so far.

A `decimal` stores numbers as a sign bit (positive or negative) and a pair of integers. There's a 96-bit integer, and the value of the `decimal` is this first integer (negated if the sign bit says so) divided by 10 raised to the power of the second integer, which is a number in the range of 0 to 28.⁵ Ninety-six bits is enough to represent any 28-digit decimal integer (and some, but not all, 29-digit ones), so the second integer—the one representing the power of 10 by which the first is divided—effectively says where the decimal point goes. This format makes it possible to represent any decimal with 28 or fewer digits accurately.

When you write a literal numeric value, you can choose the type, or you can let the compiler pick a suitable type for you. If you write a plain integer, such as `123`, its type will be `int`, `uint`, `long`, or `ulong`—the compiler picks the first type from that list with a range that contains the value. (So `123` would be an `int`, `3000000000` would be a `uint`, `5000000000` would be a `long`, etc.) If you write a number with a decimal point, such as `1.23`, its type is `double`.

If you're dealing with large numbers, it's very easy to get the number of zeros wrong. This is usually bad and possibly very expensive or dangerous, depending on your application area. C# provides some mitigation by allowing you to add underscores anywhere in numeric literals, to break the numbers up however you please. This is analogous to the common practice in most English-speaking countries of using a comma to separate zeros into groups of three. For example, instead of writing `5000000000`, most native English speakers would write `5,000,000,000`, instantly making it much easier to see that this is 5 billion and not, say, 50 billion, or 500 million. (What many native English speakers don't know is that several countries around the world use a period for this and would write `5.000.000.000` instead, using the comma where most native English speakers would put a decimal point. Interpreting a value such as €100.000 requires you to know which country's conventions are in use if you don't want to make a disastrous financial miscalculation. But I digress.) In C# we can do something similar by writing the numeric literal as `5_000_000_000`.

You can tell the compiler that you want a specific type by adding a suffix. So `123U` is a `uint`, `123L` is a `long`, and `123UL` is a `ulong`. Suffix letters

are case- and order-independent, so instead of `123UL`, you could write `123Lu`, `123uL`, or any other permutation. For `double`, `float`, and `decimal`, use the `D`, `F`, and `M` suffixes, respectively.

These last three types all support a decimal exponential literal format for large or small numbers, where you put a decimal, then the letter `E` followed by an integer. The value is the first number multiplied by 10 raised to the power of the second. For example, the literal value `1.5E-20` is the value 1.5 multiplied by 10^{-20} . (This happens to be of type `double`, because that's the default for a number with a decimal point, regardless of whether it's in exponential format. You could write `1.5E-20F` and `1.5E-20M` for `float` and `decimal` constants with equivalent values.)

It's often useful to be able to write integer literals in hexadecimal, because the digits map better onto the binary representation used at runtime. This is particularly important when different bit ranges of a number represent different things. For example, you may need to deal with a numeric error code that originated from a Windows system call—these occasionally crop up in exceptions. In some cases, these codes use the top-most bit to indicate success or failure, the next few bits to indicate the origin of the error, and the remaining bits to identify the specific error. For example, the COM error code `E_ACCESSDENIED` has the value `-2,147,024,891`. It's hard to see the structure in decimal, but in hexadecimal, it's easier: `80070005`. The `8` indicates that this is an error, and the `007` that follows indicates that this was originally a plain Win32 error that has been translated into a COM error. The remaining bits indicate that the Win32 error code was `5` (`ERROR_ACCESS_DENIED`). C# lets you write integer literals in hexadecimal for scenarios like these, where the hex representation is more readable. You just prefix the number with `0x`; therefore, in this case, you would write `0x80070005`.

You can also write binary literals by using the `0b` prefix. Digit separators can be used in hex and binary just as they can in decimals, although it's more common to group binary digits by fours, like so: `0b_0010_1010`. Obviously this makes any binary structure in a number even more evident than hexadecimal does, but 32-bit binary literals are inconveniently long, which is why we often use hexadecimal instead.

Numeric conversions

Each of the built-in numeric types uses a different representation for storing numbers in memory. Converting from one form to another requires some work—even the number 1 looks quite different if you inspect its binary representations as a `float`, an `int`, and a `decimal`. However, C# is able to generate code that converts between formats, and it will often do so automatically. [Example 2-33](#) shows some cases in which this will happen.

Example 2-33. Implicit conversions

```
int i = 42;
double di = i;
Console.WriteLine(i / 5);
Console.WriteLine(di / 5);
Console.WriteLine(i / 5.0);
```

The second line assigns the value of an `int` variable into a `double` variable. The C# compiler generates the necessary code to convert the integer value into its equivalent floating-point value. More subtly, the last two lines will perform similar conversions, as we can see from the output of that code:

```
8
8.4
8.4
```

This shows that the first division produced an integer result—dividing the integer variable `i` by the integer literal 5 caused the compiler to generate code that performs integer division, so the result is 8. But the other two divisions produced a floating-point result. In the second case, we’ve divided the `double` variable `di` by an integer literal 5. C# converts that 5 to floating point before performing the division. (As an optimization, in this particular case the compiler happens to perform that conversion at compile time, so it emits the same code for that expression as it would if we had written `di / 5.0`.) And in the final line, we’re dividing an integer variable by a floating-point literal. This time, it’s the variable’s value

that gets turned from an integer into a floating-point value before the division takes place. (Since `i` is a variable, not a constant, the compiler emits code that performs that conversion at runtime.)

In general, when you perform arithmetic calculations that involve a mixture of numeric types, C# will pick the type with the largest range and *promote* values of types with a narrower range into that larger one before performing the calculations. (Arithmetic operators generally require all their operands to have the same type, so if you supply operands with different types, one type has to “win” for any particular operator.) For example, `double` can represent any value that `int` can, and many that it cannot, so `double` is the more expressive type.⁶

C# will perform numeric conversions implicitly whenever the conversion is a promotion (i.e., the target type has a wider range than the source), because there is no possibility of the conversion failing. However, it will not implicitly convert in the other direction. The second and third lines of [Example 2-34](#) will fail to compile because they attempt to assign expressions of type `double` into an `int`, which is a *narrowing* conversion, meaning that the source might contain values that are out of the target’s range.

Example 2-34. Errors: implicit conversions not available

```
int i = 42;
int willFail = 42.0;
int willAlsoFail = i / 1.0;
```

It is possible to convert in this direction, just not implicitly. You can use a *cast*, where you specify the name of the type to which you’d like to convert in parentheses. [Example 2-35](#) shows a modified version of [Example 2-34](#), where we state explicitly that we want a conversion to `int`, and we either don’t mind that this conversion might not work correctly or we have reason to believe that, in this specific case, the value will be in range. Note that on the final line I’ve put parentheses around the expression after the cast. That makes the cast apply to the whole expression; otherwise, C#’s rules of precedence mean it would apply just to the `i` variable, and since that’s already an `int`, it would have no effect.

Example 2-35. Explicit conversions with casts

```
int i = 42;  
int i2 = (int) 42.0;  
int i3 = (int) (i / 1.0);
```

So narrowing conversions require explicit casts, and conversions that cannot lose information occur implicitly. However, with some combinations of types, neither is strictly more expressive than the other. What should happen if you try to add an `int` to a `uint`? Or an `int` to a `float`? These types are all 32 bits in size, so none of them can possibly offer more than 2^{32} distinct values, but they have different ranges, which means that each has values it can represent that the other types cannot. For example, you can represent the value 3,000,000,001 in a `uint`, but it's too large for an `int` and can only be approximated in a `float`. As floating-point numbers get larger, the values that can be represented get farther apart—a `float` can represent 3,000,000,000 and also 3,000,001,024 but nothing in between. So for the value 3,000,000,001, `uint` seems better than `float`. But what about -1? That's a negative number, so `uint` can't cope with that. Then there are very large numbers that `float` can represent that are out of range for both `int` and `uint`. Each of these types has its strengths and weaknesses, and it makes no sense to say that one of them is generally better than the rest.

Surprisingly, C# allows some implicit conversions even in these potentially lossy scenarios. The rules consider only range, not precision: implicit conversions are allowed if the target type's range completely contains the source type's range. So you can convert from either `int` or `uint` to `float`, because although `float` is unable to represent some values exactly, there are no `int` or `uint` values that it cannot at least approximate. But implicit conversions are not allowed in the other direction, because there are some `float` values that are simply too big—unlike `float`, the integer types can't offer approximations for bigger numbers.

You might be wondering what happens if you force a narrowing conversion to `int` with a cast, as [Example 2-35](#) does, but in situations where the number is out of range. The answer depends on the type from which you

are casting. Conversion from one integer type to another works differently than conversion from floating point to integer. In fact, the C# specification does not define how floating-point numbers that are too big should be converted to an integer type—the result could be anything. But when casting between integer types, the outcome is well defined. If the two types are of different sizes, the binary will be either truncated or padded with zeros (or ones, if the source type is signed and the value is negative) to make it the right size for the target type, and then the bits are just treated as if they are of the target type. This is occasionally useful but can more often produce surprising results, so you can choose an alternative behavior for any out-of-range cast by making it a *checked* conversion.

Checked contexts

C# defines the `checked` keyword, which you can put in front of either a block statement or an expression, making it a *checked context*. This means that certain arithmetic operations, including casts, are checked for range overflow at runtime. If you cast a value to an integer type in a checked context and the value is too high or low to fit, an error will occur—the code will throw a `System.OverflowException`. (As you'll see in [Chapter 4](#), C# 11.0's *generic math* feature provides some other ways to deal with out-of-range casts, but checked contexts continue to be an important concept when using generic math.)

As well as checking casts, a checked context will detect range overflows in ordinary arithmetic. Addition, subtraction, and other operations can take a value beyond the range of its data type. For integers, this causes the value to “roll over” when unchecked, so adding 1 to the maximum value produces the minimum value, and vice versa for subtraction.

Occasionally, this wrapping can be useful. For example, if you want to determine how much time has elapsed between two points in the code, one way to do this is to use the `Environment.TickCount` property.⁷ (This is more reliable than using the current date and time, because that can change as a result of the clock being adjusted or when moving between time zones. The tick count just keeps increasing at a steady rate. That said, in real code you'd probably use the runtime libraries' `Stopwatch` class.)

[Example 2-36](#) shows one way to do this.

Example 2-36. Exploiting unchecked integer overflow

```
int start = Environment.TickCount;  
DoSomeWork();  
int end = Environment.TickCount;  
  
int totalTicks = end - start;  
Console.WriteLine(totalTicks);
```

The tricky thing about `Environment.TickCount` is that it occasionally “wraps around.” It counts the number of milliseconds since the system last rebooted, and since its type is `int`, it will eventually run out of range. A span of 25 days is 2.16 billion milliseconds—too large a number to fit in an `int`. (You could avoid this by using the `TickCount64` property, which is good for almost 300 million years. But this is unavailable in .NET Framework.) Imagine the tick count is 2,147,483,637, which is 10 short of the maximum value for `int`. What would you expect it to be 100 ms later? It can’t be 100 higher (2,147,483,727), because that’s too big a value for an `int`. We’d expect it to get to the highest possible value after 10 ms, so after 11 ms, it’ll roll round to the minimum value; thus, after 100 ms, we’d expect the tick count to be 89 above the minimum value (which would be -2,147,483,559).

WARNING

The tick count is not necessarily precise to the nearest millisecond in practice. It often stands still for milliseconds at a time before leaping forward in increments of 10 ms, 15 ms, or even more. However, the value still rolls over—you just might not be able to observe every possible tick value as it does so.

Interestingly, [Example 2-36](#) handles this perfectly. If the tick count in `start` was obtained just before the count wrapped, and the one in `end` was obtained just after, `end` will contain a much lower value than `start`, which seems upside down, and the difference between them will be large—larger than the range of an `int`. However, when we subtract `start` from `end`, the overflow rolls over in a way that exactly matches the way the tick count rolls over, meaning we end up getting the correct

result regardless. For example, if the `start` contains a tick count from 10 ms before rollover, and `end` is from 90 ms afterward, subtracting the relevant tick counts (i.e., subtracting $-2,147,483,558$ from $2,147,483,627$) seems like it should produce a result of $4,294,967,185$. But because of the way the subtraction overflows, we actually get a result of 100, which corresponds to the elapsed time of 100 ms.

But in most cases, this sort of integer overflow is undesirable. It means that when dealing with large numbers, you can get results that are completely incorrect. A lot of the time, this is not a big risk, because you will be dealing with fairly small numbers, but if there is any possibility that your calculations might encounter overflow, you might want to use a checked context. Any arithmetic performed in a checked context will throw an exception when overflow occurs. You can request this in an expression with the `checked` operator, as [Example 2-37](#) shows. Everything inside the parentheses will be evaluated in a checked context, so you'll see an `OverflowException` if the addition of `a` and `b` overflows. The `checked` keyword does not apply to the whole statement here, so if an overflow happens as a result of adding `c`, that will not cause an exception.

Example 2-37. Checked expression

```
int result = checked(a + b) + c;
```

You can also turn on checking for an entire block of code with a `checked` statement, which is a block preceded by the `checked` keyword, as [Example 2-38](#) shows. Checked statements always involve a block—you cannot just add the `checked` keyword in front of the `int` keyword in [Example 2-37](#) to turn that into a checked statement. You'd also need to wrap the code in braces.

Example 2-38. Checked statement

```
checked
{
    int r1 = a + b;
```

```
int r2 = r1 - (int) c;  
}
```

WARNING

A `checked` block only affects the lines of code inside the block. If the code invokes any methods, those will be unaffected by the presence of the `checked` keyword—there isn't some *checked* bit in the CPU that gets enabled on the current thread inside a `checked` block. (In other words, this keyword's scope is lexical, not dynamic.)

C# also has an `unchecked` keyword. You can use this inside a `checked` block to indicate that a particular expression or nested block should not be a checked context. This makes life easier if you want everything except for one particular expression to be checked—rather than having to label everything except the chosen part as checked, you can put all the code into a `checked` block and then exclude the one piece that wants to allow overflow without errors.

You can configure the C# compiler to put everything into a checked context by default, so that only explicitly `unchecked` expressions and statements will be able to overflow silently. In Visual Studio, you can configure this by opening the project properties, going to the Build section's Advanced subsection. Or you can edit the `.csproj` file, adding `<CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>` inside a `<PropertyGroup>`. Be aware that there's a significant cost—checking can make individual integer operations several times slower. The impact on your application as a whole will be smaller, because programs don't spend their whole time performing arithmetic, but the cost may still be nontrivial. Of course, as with any performance matter, you should measure the practical impact. You may find that the performance cost is an acceptable price to pay for the guarantee that you will find out about unexpected overflows.

BigInteger, Int128, UInt128, and Half

The preceding sections discussed the various numeric types that get native support from the runtime, but there are some other numeric types worth being aware of: there is `BigInteger` (in the `System.Numerics` namespace), and also `Int128`, `UInt128`, and `Half` (in the `System` namespace). These are part of the runtime libraries and get no special recognition from the C# compiler, so they don't strictly belong in this section of the book. However, they define arithmetic operators and conversions, meaning that you can use them just like the built-in data types.

They will compile to slightly less compact code—the compiled format for .NET programs can represent the integer and floating-point types described earlier natively, but these other types have to rely on the more general-purpose mechanisms used by ordinary class library types. In theory they are likely to be significantly slower too, although in an awful lot of code, the speed at which you can perform basic arithmetic on small integers is not a limiting factor, so it's quite possible that you won't notice. And as far as the programming model goes, these types look and feel like normal numeric types in your code.

The `Int128` and `UInt128` types are similar to the smaller integer types already seen, but since they are stored as 128-bit values, they offer a much wider numeric range. `BigInteger`, as the name suggests, also represents an integer, but it will grow as large as is necessary to accommodate values. So unlike the built-in numeric types, it has no theoretical limit on its range. [Example 2-39](#) uses it to calculate values in the Fibonacci sequence, showing every 100,000th value. This quickly produces numbers far too large to fit into any of the other integer types. I've shown the full source of this example, including the `using` directive, to illustrate that this type is defined in the `System.Numerics` namespace.

Example 2-39. Using `BigInteger`

```
using System.Numerics;

BigInteger i1 = 1;
BigInteger i2 = 1;
```

```

Console.WriteLine(i1);
int count = 0;
while (true)
{
    // The % operator returns the remainder of dividing its 1st operand by its
    // 2nd, so this displays the number only when count is divisible by 100000.
    if (count++ % 100000 == 0)
    {
        Console.WriteLine(i2);
    }
    BigInteger next = i1 + i2;
    i1 = i2;
    i2 = next;
}

```

Although `BigInteger` imposes no fixed limit, there are practical limits. You might produce a number that's too big to fit in the available memory, for example. Or more likely, the numbers may grow large enough that the amount of CPU time required to perform even basic arithmetic becomes prohibitive. But until you run out of either memory or patience, `BigInteger` will grow to accommodate numbers as large as you like.

The runtime libraries also offer `System.Half`, a 16-bit IEEE 754 floating-point format offering 10 bits of precision and a magnitude range from 5.96×10^{-8} to 65,504. This precision is only slightly better than three-decimal digits, so `Half` is typically useful only in applications where reducing memory consumption is more important than precision. That can be common in artificial intelligence applications, and also in games and some kinds of media processing. Although this type is supported extensively throughout the runtime libraries (for example, JSON serialization supports it, and all the other numeric types offer conversion to and from `Half`) the fact that it is currently just a library feature with no intrinsic runtime support means it can't currently make use of the hardware support for this format that some CPUs offer (although to be fair, very few CPU architectures support this format today). `Half` implements arithmetic by first converting its value to `float`, which is somewhat inefficient. That could conceivably change, but for now, this type's main purpose is to enable natural support for this representation in libraries for systems that can work with it directly. For example, this is a common format when performing calculations on a graphics card. Your C# code

might create an array of `Half` values and then send them to the graphics hardware to be processed by the GPU. In this scenario, the relatively weak performance the CLR offers for `Half` doesn't matter, because the GPU is doing the heavy number crunching.

Booleans

C# defines a type called `bool`, or as the runtime calls it, `System.Boolean`. This offers only two values: `true` and `false`. Whereas some C-family languages allow numeric types to stand in for Boolean values, with conventions such as 0 meaning false and anything else meaning true, C# will not accept a number. It demands that values indicating truth or falsehood be represented by a `bool`, and none of the numeric types is convertible to `bool`. For example, in an `if` statement, you cannot write `if (someNumber)` to get some code to run only when `someNumber` is nonzero. If that's what you want, you need to say so explicitly by writing `if (someNumber != 0)`.

Strings and Characters

The `string` type (synonymous with the CLR `System.String` type) represents text. A string is a sequence of values of type `char` (or `System.Char`, as the CLR calls it), and each `char` is a 16-bit value representing a single UTF-16 *code unit*.

A common mistake is to think that each `char` represents a character. (The type's name has to share some of the blame for this.) It's often true, but not always. There are two factors to bear in mind: first, something that we might think of as a single character can be made up from multiple Unicode *code points*. (The code point is Unicode's central concept, and in English at least, each character is represented by a single code point, but some languages are more complex.) [Example 2-40](#) uses Unicode's 0301 "COMBINING ACUTE ACCENT" to add an accent to a letter to form the text `cafés`.

Example 2-40. Characters versus `char`

```
char[] chars = ['c', 'a', 'f', 'e', (char) 0x301, 's'];  
string text = new string(chars);
```

So this string is a sequence of six `char` values, but it represents text that seems to contain just five characters. There are other ways to achieve this—I could have used code point 00E9 “LATIN SMALL LETTER E WITH ACUTE” to represent that accented character as a single code point. But either approach is valid, and there are plenty of scenarios in which the only way to create the exact character required is to use this combining character mechanism. This means that certain operations on the `char` values in a string can have surprising results—if you were to reverse the order of the values, the resulting string would not look like a reversed version of the text—the acute accent would now apply to the s, resulting in `śefac`! (If I had used 00E9 instead of combining e with 0301, reversing the characters would have produced the less surprising `séfac`.)

Unicode’s combining marks notwithstanding, there is a second factor to consider. The Unicode standard defines more code points than can be represented in a single 16-bit value. (We passed that point back in 2001, when Unicode 3.1 defined 94,205 code points.) UTF-16 represents any code point with a value higher than 65,535 as a pair of UTF-16 code units, referred to as a *surrogate pair*. The Unicode standard defines rules for mapping code points to surrogate pairs in a way that the resulting code units have values in the range 0xD800 to 0xDFFF, a reserved range for which no code points will ever be defined. (For example, code point 10C48, “OLD TURKIC LETTER ORKHON BASH,” which looks like , would become 0xD803, followed by 0xDC48.)

In summary, items that users perceive as single characters might be represented with multiple Unicode code points, and some single code points might be represented as multiple code units. Manipulating the individual `char` values that make up a `string` is therefore a job you should approach with caution. Often, a simple approach works well enough—if, for example, you want to search a string for some specific character that you know fits in a single code unit (such as `/`), a simple `char`-based search

will work perfectly well. However, if you have a more complex scenario that requires you to detect all multi-code-unit sequences correctly, the runtime libraries offer some help here.

The `string` type offers an `EnumerateRunes` method that effectively combines surrogate pairs back into the value of the code point they represent. It presents the string as a sequence of values of type `Rune`, and if a string contained the `0xD803, 0xDC48` sequence just described, this pair of `char` values would be presented as a single `Rune` with the value `0x10C48`. The `Rune` type still operates at the level of individual code points, so it won't help you with combining characters, but if you need to go to that next level, the runtime libraries define a `StringInfo` class in the `System.Globalization` namespace. This interprets a string as a sequence of "text elements," and in cases such as `cafés`, it will report the `é` as a single text element, even when it was formed with two code points using the combining character mechanism.

Immutability of strings

.NET strings are immutable. There are many operations that sound as though they will modify a string, such as concatenation, or the `ToUpper` and `ToLower` methods offered by instances of the `string` type, but each of these generates a new string, leaving the original one unmodified. This means that if you pass strings as arguments, even to code you didn't write, you can be certain that it cannot change your strings.

The downside of immutability is that string processing can be inefficient. If you need to do work that performs a series of modifications to a string, such as building it up character by character, you will end up allocating a lot of memory, because you'll get a separate string for each modification. This creates a lot of extra work for .NET's garbage collector, causing your program to use more CPU time than necessary. In these situations, you can use a type called `StringBuilder`. (This type gets no special recognition from the C# compiler, unlike `string`.) This is conceptually similar to a `string`—it is a sequence of `char` values and offers various useful string manipulation methods—but it is modifiable. Alternatively, in extremely performance-sensitive scenarios, you might use the techniques shown in [Chapter 18](#).

String manipulation methods

The `string` type has numerous instance methods for working with strings. I already mentioned `ToUpper` and `ToLower`, but there are also methods for finding text within the string, including `IndexOf` and `LastIndexOf`. `StartsWith` and `EndsWith` return a `bool` indicating whether the string starts or ends with a particular character or string. `Split` takes one or more separator characters (e.g., commas or spaces) and returns an array with an entry for each substring between the separators. For example, `"One,two,three".Split(',')` returns an array containing the three strings `"One"`, `"two"`, and `"three"`. `Substring` takes a starting position and optional length and returns a new string containing all characters from the start position up to either the end of the string or the specified length; `Remove` does the opposite: it forms a new string by removing the part of the original string that `Substring` would have returned. `Insert` forms a new string by inserting one string into the middle of another. `Replace` returns a new string formed by replacing all instances of a particular character or string with another. `Trim` can be used to remove unwanted leading and trailing characters such as whitespace.

Formatting data in strings

C# provides a syntax that makes it easy to produce strings containing a mixture of fixed text and information determined at runtime. (The official name for this feature is *string interpolation*.) For example, if you have local variables called `name` and `age`, you could use them in a string, as [Example 2-41](#) shows.

Example 2-41. Expressions in strings

```
string message = $"{name} is {age} years old";
```

When you put a `$` symbol in front of a string literal, the C# compiler looks for embedded expressions delimited by braces and produces code that will insert a textual representation of the expression at that point in the string. (So if `name` and `age` were `Ian` and `50`, respectively, the

string's value would be "Ian is 50 years old" .) As [Example 2-42](#) shows, embedded expressions can be more complex than just variable names.

Example 2-42. More complex expressions in strings

```
double width = 3, height = 4;  
string info = $"Hypotenuse: {Math.Sqrt(width * width + height * height)}";
```

If a string contains complex expressions, it might become a bit too large to fit on a single line. As [Example 2-43](#) shows, you can split interpolated strings or string literals over multiple lines by breaking them up and using `+` to join the parts together. (Visual Studio can do this for you: if you put the text cursor where you'd like to split the string and press Enter, it automatically adds the necessary extra `"` and `+` symbols.)

Example 2-43. Splitting up an interpolated string

```
string info = $"Hypotenuse:" +  
    $" {Math.Sqrt(width * width + height * height)}";
```

The `+` symbol denotes concatenation, and in general the compiler implements this with the `string` type's `Concat` method. However, in this case it will see that we're just using it to split an interpolated string across multiple lines, and it will generate exactly the same code as it would if it were a single interpolated string. This is important because if it generated the two parts separately and then concatenated them, it would create more string objects than necessary, increasing memory usage at runtime.

If an individual expression is complex, you might also want to split that across multiple lines. This wasn't possible before C# 11.0: unless you're writing a *verbatim string* (described in the next section) string literals aren't allowed to contain new lines, and C# used to impose this rule even on the expressions in an interpolated string. C# 11.0 relaxed this restriction, enabling us to split individual expressions in an interpolated string across multiple lines, as [Example 2-44](#) shows. (If you want line breaks in

the actual text content, you will still need to use either verbatim strings or raw string literals, which I will describe shortly.)

Example 2-44. Splitting an interpolated expression across lines

```
string info = $"Hypotenuse: {Math.Sqrt(  
    width * width +  
    height * height)}";
```

If you want to use string interpolation but you also want the resulting string to include opening or closing braces, you double them up. When an interpolated string contains either `{{` or `}}`, the compiler does not interpret them as delimiting embedded expressions and just produces a single `{` or `}` in the output. For example, `$"Brace: {{, braces: {{}}, width: {width}, braced width: {{{width}}}"` evaluates to `Brace: {, braces: {}, width: 3, braced width: {3}` (assuming `width` is 3 here).

The runtime libraries offer another mechanism for plugging values into a string called *composite formatting*. The `string` class's `Format` method takes a string with numbered placeholders of the form `{0}` and `{1}`, followed by a list of arguments supplying the values for these placeholders.

[Example 2-45](#) uses this to achieve the same effect as Examples [2-41](#) and [2-42](#).

Example 2-45. Using `string.Format`

```
string message = string.Format("{0} is {1} years old", name, age);  
string info = string.Format(  
    "Hypotenuse: {0}",  
    Math.Sqrt(width * width + height * height));
```

This composite formatting mechanism is older—it has been around since C# 1.0, whereas string interpolation was introduced in C# 6.0—so you will see it cropping up in quite a few places. `Console.WriteLine` supports it, for example. In most cases, it's not the preferred approach. String interpolation is much less error prone—composite formatting uses position-based placeholders, making it all too easy to put an expression in the

wrong place. It's also tedious for anyone reading the code to try and work out how the numbered placeholders relate to the arguments that follow, particularly as the number of expressions increases. Interpolated strings are usually much easier to read. However, composite formatting has one important capability that interpolated strings do not: you can choose the format string at runtime. This is widely used for internationalization: applications can choose which format string to use based on the preferred language the user has configured.

Interpolated strings can sometimes offer performance benefits. If we select a composite format string at runtime, all the processing associated with that string must also be done at runtime, but with string interpolation, the compiler may be able to perform compile-time optimizations. For example, if an expression in an interpolated string is a `const` string ([Chapter 3](#) describes the `const` keyword), the compiler will insert its value into the string at compile time. Furthermore, libraries may indicate that they want to be involved in the interpolation process, making it possible to avoid ever creating a string in cases where that string won't be used. When might you write an interpolated string that won't be used? Look at [Example 2-46](#).

Example 2-46. A potentially unused interpolated string

```
Debug.Assert(everythingIsOk, $"Everything is *not* OK: {myApplicationModel}");
```

This uses `Debug.Assert`, a diagnostic method you can add to your code to detect when your application has got into some unexpected state. `Debug.Assert` checks its first argument, and if that's `false`, it will halt the program, displaying the message passed as the second argument. But if the first argument is `true`, it proceeds without ever using the second argument. In this example, if calling `ToString()` on the `myApplicationModel` in the interpolated string were expensive, it would be bad news if that ran even in cases where everything is in fact OK—our program might be doing a great deal of work to create a string that gets thrown away. But [Example 2-46](#) uses an overload of `Debug.Assert` that takes advantage of string interpolation's ability to avoid ever creating that string in cases where it won't be used. This same mechanism could be used by

logging frameworks, in which it's common for code to be able to generate a lot of strings to provide detailed descriptions of what's happening but which will be unused in the typical case where verbose logging has not been enabled.

Interpolated strings have another performance benefit: the work of finding all the braces to determine which parts of the string are plain text and which are value placeholders can be done at compile time, so even when a string does get created, interpolation has an advantage over composite formatting. If you really do need to select composite format strings at runtime (e.g., to support internationalization) you can't avoid doing runtime processing. However, .NET 8.0 adds a new type, `CompositeFormat`, that can avoid *repeating* work in cases where we use the same format string repeatedly. [Example 2-47](#) obtains a format string from a method (`GetPersonFormat`; not shown here, but it could look up the text based on the locale). `CompositeFormat.Parse` will parse this string to find the placeholders. Each time the loop calls `string.Format`, that can use this information instead of having to rescan the format string every time. This is not quite as efficient as string interpolation, which does that work at compile time, but in cases where you use the same composite format string frequently, this avoids repeating work unnecessarily.

Example 2-47. Using `CompositeFormat`

```
CompositeFormat nameAgeFormat = CompositeFormat.Parse(GetPersonFormat());

foreach (Person p in people)
{
    string message = string.Format(
        CultureInfo.InvariantCulture, nameAgeFormat, p.Name, p.Age);
    Console.WriteLine(message);
}
```

With some data types, there are choices to be made about their textual representation. For example, with floating-point numbers, you might want to limit the number of decimal places, or force the use of exponential notation. (For example, `1e6` instead of `1000000`.) In .NET, we control this with a *format specifier*, which is a string describing how to convert

some data to a string. Some data types have only one reasonable string representation, so they do not support this, but with types that have multiple string forms, you can pass the format specifier as an argument to the `ToString` method. For example, `System.Math.PI.ToString("f4")` formats the `PI` constant (which is of type `double`) to four decimal places (`"3.1416"`). There are nine built-in formats for numbers, and if none of those suits your requirements, there is also a minilanguage for defining custom formats. Moreover, different types use different format strings—dates work quite differently from numbers, for example—so the full range of available formats is too large to list here. Microsoft supplies extensive documentation of the details.

When using composite formatting, you can include a format specifier in the placeholder; for example, `{0:f3}` indicates that the first expression is to be formatted with three digits after the decimal point. You can include a format specifier in a similar way with string interpolation.

[Example 2-48](#) shows the age with one digit after the decimal point.

Example 2-48. Format specifiers

```
string message = $"{name} is {age:f1} years old";
```

There's one wrinkle with this: with many data types, the process of converting to a string is culture-specific. For example, as mentioned earlier, in the US and the UK, decimals are typically written with a period between the whole number part and the fractional part, and you might use commas to group digits for readability, but some European countries invert this: they use periods to group digits, while the comma denotes the start of the fractional part. So what might be written as 1,000.2 in one country could be written as 1.000,2 in another.

As far as numeric literals in source code are concerned, this is a nonissue: C# uses underscores for digit grouping and always uses a period as the decimal point. But what about processing numbers at runtime? By default, you will get conventions determined by the current thread's culture, and unless you've changed that, it will use the regional settings of the computer. Sometimes this is useful—it can mean that numbers, dates,

and so on are correctly formatted for whatever locale a program runs in. However, it can be problematic: if your code relies on strings being formatted in a particular way (to serialize data that will be transmitted over a network, for example), you may need to apply a particular set of conventions. For this reason, you can pass the `string.Format` method a *format provider*, an object that controls formatting conventions. Likewise, data types with culture-dependent representations accept an optional format provider argument in their `ToString` methods. But how do you control this when using string interpolation? There's nowhere to put the format provider. You can solve this with the `string` type's `Create` method, as shown in [Example 2-49](#).

Example 2-49. Format specifiers with invariant culture

```
decimal v = 1234567.654m;  
string i = string.Create(CultureInfo.InvariantCulture, $"Quantity {v:N}");  
string f = string.Create(new CultureInfo("fr"), $"Quantity {v:N}");  
string frc = string.Create(new CultureInfo("fr-FR"), $"Quantity {v:C}");  
string cac = string.Create(new CultureInfo("fr-CA"), $"Quantity {v:C}");
```

This passes various different format providers to the `string.Create` method, but it uses only a couple of different interpolated strings. Notice that it puts `:N` after the variable name in the first two lines. This asks for normal numeric formatting, including digit separators. The first call passes the *invariant culture*, which guarantees consistent formatting regardless of the locale in which the code runs, with the effect that `i` gets the value `"Quantity 1,234,567.654"`. The third line uses a `CultureInfo` object constructed with the argument `"fr"`. This tells it that we want it to format strings in the ways typically expected in French-speaking cultures, so the `f` variable gets the value `"Quantity 1.234.567,654"`. The final two lines use `:C`, indicating we'd like to show the value as a currency. I've passed cultures representing France and the French-speaking parts of Canada, resulting in euro and dollar symbols, respectively.

It may seem odd that this works: normally, method arguments are evaluated before being passed into the method, so you might expect the interpolated string to be turned into a normal string before the call to

`string.Create`, meaning it would be too late to apply the specified format provider. But as I said earlier, methods can indicate that they want to be involved in the string interpolation process. This `string.Create` method does exactly that, enabling it to take control of the process, which is how it is able to apply the format provider.

Verbatim string literals

C# supports an alternative way of expressing a string value: you can prefix a string literal with the `@` symbol like so: `@"Hello"`. Strings of this form are called *verbatim string literals*. They are useful for two reasons: they can improve the readability of strings containing backslashes, and they make it possible to write multiline string literals. (The newer *raw string literal* feature described in the next section can also do this, and is more flexible, but verbatim literals can sometimes be simpler. They've also been around for longer so you are likely to come across them in existing codebases.)

In a normal string literal, the compiler treats a backslash as an escape character, enabling various special values to be included. For example, in the literal `"Hello\tWorld!"` the `\t` denotes a single tab character (code point 9). This is a common way to express control characters in C-family languages. You can also use the backslash to include a double quote in a string—the backslash prevents the compiler from interpreting the character as the end of the string. Useful though this is, it makes including a backslash in a string a bit awkward: you have to write two of them. Since Windows uses backslashes in paths, this can get ugly:

`"C:\\Windows\\System32\\"`. A verbatim string literal can be useful here, because it treats backslashes literally, enabling you to write just `@"C:\Windows\System32"`. (You can still include double quotes in a verbatim literal: just write two double quotes in a row. For example, `@"Hello ""World"""` produces the string value `Hello "World"`.)

You can use `@` in front of an interpolated string. This combines the benefits of verbatim literals—straightforward use of backslashes and newlines—with support for embedded expressions.

Verbatim string literals also allow values to span multiple lines. With a normal string literal, the compiler will report an error if the closing double quote is not on the same line as the opening one. But with a verbatim string literal, the string can cover as many lines of source as you like.

The resulting string will use whichever line-ending convention your source code uses. Just in case you've not encountered this, one of the unfortunate accidents of computing history is that different systems use different character sequences to denote line endings. The predominant system in internet protocols is to use a pair of control codes for each line end: in either Unicode or ASCII, we use code points 13 and 10, denoting a *carriage return* and a *line feed*, respectively, often abbreviated to CR LF. This is an archaic hangover from the days before computers had screens, and starting a new line meant moving the teletype's print head back to its start position (carriage return) and then moving the paper up by one line (line feed). Anachronistically, the HTTP specification requires this representation, as do the various popular email standards, SMTP, POP3, and IMAP. It is also the standard convention on Windows. Unfortunately, the Unix operating system does things differently, as do most of its derivatives and lookalikes such as macOS and Linux—the convention on these systems is to use just a single line feed character. The C# compiler accepts either and will not complain if a single source file contains a mixture of both conventions. This introduces a potential problem for multiline string literals if you are using a source control system that converts line endings for you. For example, *Git* is a very popular source control system, and thanks to its origins (it was created by Linus Torvalds, who also created Linux), there is a widespread convention of using Unix-style line endings in its repositories. However, on Windows it can be configured to convert working copies of files to a CR LF representation, automatically converting them back to LF when committing changes. This means that files will appear to use different line-ending conventions depending on whether

you're looking at them on a Windows system or a Unix one. (And it might even vary from one Windows system to another, because the default line-ending handling is configurable. Individual users can configure the machine-wide default setting and can also set the configuration for their local clone of any repository if the repository does not specify the setting itself.) This in turn means that compiling a file containing a multiline verbatim string literal on a Windows system could produce subtly different behavior than you'd see with the exact same file on a Unix system, if automatic line-ending conversion is enabled (which it is by default on most Windows installations of Git). That might be fine—you typically want CR LF when running on Windows and LF on Unix—but it could cause surprises if you deploy code to a machine running a different OS than the one you built it on. So it's important to provide a *.gitattributes* file in your repositories so that they can specify the required behavior, instead of relying on changeable local settings. If you need to rely on a particular line ending in a string literal, it's best to make your *.gitattributes* disable line-ending conversions.

Raw string literals

Certain text formats can be awkward to produce with string literals. JSON is particularly tricky because it tends to contain lots of braces and double quotes. The safest way to produce JSON is with the `System.Text.Json` API described in [Chapter 15](#), so for most production code I would advise you not to use string literals at all when creating JSON. However, there are some situations (especially test code) where the risks of creating malformed JSON are low, in which case a string literal might be the best choice. However, this can lead to horrors of the kind shown in [Example 2-50](#).

Example 2-50. Messy JSON string interpolation

```
static string MakeRightAngledTriangleJson(double width, double height)
{
    // Wonky indentation is necessary here to get correctly indented output
    return @$"{{
        ""width"": {width},
        ""height"": {height},
```

```

        ""calculated"": {{
            ""hypotenuse"": {Math.Sqrt(width * width + height * height)},
            ""area"": {width * height / 2}
        }}
    }}";
}

```

I've used a verbatim string literal here so that the JSON being produced can run over multiple lines with indentation, but there are several problems here. JSON is a format that uses a lot of double quotes and braces. Since the `"` character delimits the string, I've had to write `""` every time I want my string to contain a double quote. And since I'm using string interpolation, I've had to double up the braces anywhere I want to emit an actual brace.

There's a more subtle problem with indentation. When a statement crosses multiple lines in C#, we would normally indent it so that it's easier to see where each statement ends. But here, the content of my string is mostly aligned with the `return` statement, and the final line of the string is actually to the left of the `return` statement it belongs to, making the final line in the method look like it's the result of a cut and paste mishap. Unfortunately, it has to look like this. If I had indented the string so that it was positioned in the way we normally format C# code, all the extra whitespace would have been included in the string's value, meaning I would no longer be producing the JSON text I want. It's not too bad in this example, but as code becomes more deeply nested inside flow control structures or type definitions, we typically indent it further. This makes verbatim strings look more and more out of place, because they need to remain anchored to the left to avoid adding unwanted whitespace to the string value.

C# 11.0 added a new string literal syntax that addresses all of these problems. It is called the *raw string* syntax, and [Example 2-51](#) shows how to use it in place of the preceding example's verbatim string.

Example 2-51. JSON with an interpolated raw string literal

```

static string MakeRightAngledTriangleJson(double width, double height)
{

```

```

return $$`
{
  "width": {{width}},
  "height": {{height}},
  "calculated": {
    "hypotenuse": {{Math.Sqrt(width * width + height * height)}},
    "area": {{width * height / 2}}
  }
}
`;
}

```

With this syntax, I can write the double quote and brace characters that show up so often in JSON exactly as they will appear in the output. I've also been able to indent the entire string literal. If I had done that with a verbatim string, that extra indentation would have become part of the string, but not so here. This produces exactly the same output as before. For example, if I call this method with a `width` and `height` of 3 and 5 I get this:

```

{
  "width": 3,
  "height": 4,
  "calculated": {
    "hypotenuse": 5,
    "area": 6
  }
}

```

The opening and closing braces are not indented at all, but I still get all of the necessary indentation within the JSON. That's exactly what I wanted, but how did the compiler know to include some but not all of the white-space? When you write a multiline raw string literal, you're required to put the start of the string on its own line. (If text follows the `"""` on the same line, that means you're writing a single-line string, in which case the compiler won't allow embedded new lines.) The closing `"""` must also appear on its own line, and the compiler looks at its indentation and removes that much indentation from the start of each line of the string. Since my closing `"""` is aligned with the opening and closing braces in this string, those end up with no indentation at all in the resulting string.

(The extra line endings that the syntax requires at the start and end are not included in the resulting string. The very first character of this string is the opening `{`, not a new line; the final character is the closing `}` and this is not followed by a new line, despite how it looks in the example. If you want leading and trailing new lines, you would add blank lines at the start and end of the string.)

You might be wondering why I needed two `$` symbols. In fact, I could have written just `$""`, but then I would have needed to double up all the braces that weren't meant to be interpolated expressions. The number of `$` symbols tells the compiler how many braces I will be writing if I want to embed an expression instead of just producing an actual `{` or `}`. Since [Example 2-51](#) starts with two `$` symbols, C# will interpret individual braces as normal characters. Only when braces come in pairs will it treat them as expression delimiters. If for some reason I wanted to produce a string that contained a lot of double braces, I could start the string with `$$$`, telling the compiler that it should interpret `{` and `{{` literally, and that only if I write `{{{` am I embedding an expression. I can also omit the `$` signs entirely—if I start a raw string with just `"""` that means that I don't want string interpolation at all.

Just as we can vary the number of `$` signs, we can also use more than three double quotes. If you wanted to produce a string that contains three double quotes in a row, you can use four quotes in the delimiters. The compiler defines no fixed limit on the number of `$` or `"` signs, so no matter how many consecutive double quotes or braces you'd like to put in a string, raw string literals work without any of the messy escaping that normal or verbatim string literals require.

UTF-8 string literals

There's one last way to write a string literal: if you write `u8` after the closing `"`, this tells the compiler that you want the text as a sequence of bytes encoded as UTF-8. This can be useful when working directly with text that is encoded as UTF-8. Most JSON text is in that format, for example, and when you're using the `System.Text.Json` API described in [Chapter 15](#), it can be more efficient to avoid forcing the API to convert between the UTF-16 representation that .NET uses and the UTF-8 encoding

your actual data is using. [Example 2-52](#) uses this API to retrieve some properties from a `root` element representing a JSON document.

([Chapter 15](#) shows how to obtain such a `JsonElement`.)

Example 2-52. Specifying JSON property names as UTF-8

```
if (root.TryGetProperty("location"u8, out JsonElement locationElement))
{
    JsonElement latitudeElement = locationElement.GetProperty("latitude"u8);
    JsonElement longitudeElement = locationElement.GetProperty("longitude"u8);
    double latitude = latitudeElement.GetDouble();
    double longitude = longitudeElement.GetDouble();
    Console.WriteLine($"Location: {locationName}: {latitude},{longitude}");
}
```

Notice how each string literal here ends in `u8`. That means that when we ask `JsonElement` to find a property with a particular name, we've supplied that name using the same text encoding as will be in use in the source JSON itself. You aren't obliged to do this—these methods have overloads that let you specify the property name as an ordinary `string`, but if you use those, you force `JsonElement` to compare the UTF-16 data in that `string` with the UTF-8 in the JSON. Comparing strings that use different encodings will always be more expensive than when they use the same representation, so in cases where you know the name of the property at compile time, a UTF-8 string constant provides an easy performance improvement.

UTF-8 string constants do not produce a `string` object, because `string` always uses UTF-16 internally, so you can't do all the things you can do with a normal `string`—methods such as `Split` or `Replace` are not available. In fact, they are not objects at all. UTF-8 literals have the type `ReadOnlySpan<byte>`, which is a way to refer to a range of bytes in memory. (So you effectively get a managed pointer into the part of the DLL where the compiler put the UTF-8 bytes.) This is a much more lightweight data type than `string` but it comes with some limitations. [Chapter 18](#) discusses span types in detail. If you need a normal `byte[]` array instead of a span, you can just call the span's `ToArray()` method.

Tuples

Tuples let you combine multiple values into a single value. The name tuple (which C# shares with many other programming languages that provide a similar feature) is meant to be a generalized version of words like *double*, *triple*, *quadruple*, and so on, but we generally call them tuples even in cases where we don't need the generality. For example, even if we're talking about a tuple with two items in it, we still call it a tuple, not a double. [Example 2-53](#) creates a tuple containing two `int` values and then displays them.

Example 2-53. Creating and using a tuple

```
(int X, int Y) point = (10, 5);  
Console.WriteLine($"X: {point.X}, Y: {point.Y}");
```

That first line is a variable declaration with an initializer. It's worth breaking this down, because the syntax for tuples makes for a slightly more complex-looking declaration than we've seen so far. Remember, the general pattern for statements of this form is as follows:

```
type identifier = initial-value;
```

That means that in [Example 2-53](#), the type is `(int X, int Y)`. So we're saying that our variable, `point`, is a tuple containing two values, both of type `int`, and that we want to refer to those as `X` and `Y`. The initializer here is `(10, 5)`. So when we run the example, it produces this output:

```
X: 10, Y: 5
```

C# 12.0 has removed a limitation that used to apply to tuple types. For years, the `using` syntax for defining type aliases did not support tuple types, but now, code such as [Example 2-54](#) is permitted. This does not define a new type—it just means that in the source file that defines the alias, we can use an alternative name to refer to the type. I introduced this *using alias* syntax in [Chapter 1](#) as a way to resolve ambiguity between two

types in different namespaces with the same simple name, but there is no ambiguity with tuple types—they don't live in a namespace, so `(int X, int Y)` always refers to one particular type. There is a different reason you might want to define an alias for a tuple type: it can make your intent clear. A descriptive name might help someone reading your code to understand more easily what the tuple is meant to represent.

Example 2-54. Defining and using an alias for a tuple type

```
using Point = (int X, int Y);  
  
Point center = (0, 0);
```

If you're a fan of `var`, you'll be pleased to know that you can specify the names in the initializer using the syntax shown in [Example 2-55](#), enabling you to use `var` instead of the explicit type. This is equivalent to [Example 2-53](#).

Example 2-55. Naming tuple members in the initializer

```
var point = (X: 10, Y: 5);  
Console.WriteLine($"X: {point.X}, Y: {point.Y}");
```

If you initialize a tuple from existing variables and you do not specify names, the compiler will presume that you want to use the names of those variables, as [Example 2-56](#) shows.

Example 2-56. Inferring tuple member names from variables

```
int x = 10, y = 5;  
var point = (x, y);  
Console.WriteLine($"X: {point.x}, Y: {point.y}");
```

This raises a stylistic question: Should tuple member names start with lowercase or uppercase letters? The members are similar in nature to properties, which we'll be discussing in [Chapter 3](#), and conventionally those start with an uppercase letter. For this reason, many people believe that tuple member names should also be uppercase. To a seasoned .NET

developer, that `point.x` in [Example 2-56](#) just looks weird. However, another .NET convention is that local variables usually start with a lower-case name. If you stick to both of these conventions, tuple name inference doesn't look very useful. Many developers choose to accept lowercase tuple member names for tuples used purely in local variables, because it enables the use of the convenient name inference feature, using the more normal casing style only for tuples that are exposed outside of a method.

Arguably it doesn't matter much, because tuple member names turn out to exist only in the eye of the beholder. First, they're optional. As [Example 2-57](#) shows, it's perfectly legal to omit them. The names just default to `Item1`, `Item2`, etc.

Example 2-57. Default tuple member names

```
(int, int) point = (10, 5);  
Console.WriteLine($"X: {point.Item1}, Y: {point.Item2}");
```

Second, the names are purely for the convenience of the code using the tuples and are not visible to the runtime. You'll have noticed that I've used the same initializer expression, `(10, 5)`, as I did in [Example 2-53](#). Because it doesn't specify names, the expression's type is `(int, int)`, which matches the type in [Example 2-57](#), but I was also able to assign it straight into an `(int X, int Y)` in [Example 2-53](#). That's because the names are essentially irrelevant—these are all the same thing under the covers. (As we'll see in [Chapter 4](#), at runtime these are all represented as instances of a type called `ValueTuple<int, int>`.) The C# compiler keeps track of the names we've chosen to use, but as far as the CLR is concerned, all these tuples just have members called `Item1` and `Item2`. An upshot of this is that we can assign any tuple into any variable with the same shape, as [Example 2-58](#) shows.

Example 2-58. Structural equivalence of tuples

```
(int X, int Y) point = (50, 3);  
(int Width, int Height) dimensions = point;  
(int Age, int NumberOfChildren) person = point;
```

This flexibility is a double-edged sword. The assignments in [Example 2-58](#) seem rather sketchy. It might conceivably be OK to assign something that represents a location into something that represents a size—there are some situations in which that would be valid. But to assign that same value into something apparently representing someone’s age and the number of children they have looks likely to be wrong. The compiler won’t stop us though, because it considers all tuples comprising a pair of `int` values to have the same type. (It’s not really any different from the fact that the compiler won’t stop you assigning an `int` variable named `age` into an `int` variable named `height`. They’re both of type `int`.)

If you want to enforce a semantic distinction, you would be better off defining custom types as described in [Chapter 3](#). Tuples are really designed as a convenient way to package together a few values in cases where defining a whole new type wouldn’t really be justified.

C# does require tuples to have an appropriate shape. You cannot assign an `(int, int)` into an `(int, string)`, nor into an `(int, int, int)`. However, all of the implicit conversions in [“Numeric conversions”](#) work, so you can assign anything with an `(int, int)` shape into an `(int, double)` or a `(double, long)`. So a tuple is really just like having a handful of variables neatly contained inside another variable.

Tuples support comparison, so you can use the `==` and `!=` relational operators described later in this chapter. To be considered equal, two tuples must have the same shape, and each value in the first tuple must be equal to its counterpart in the second tuple.

Tuple Deconstruction

Sometimes you will want to split a tuple back into its component parts. The most straightforward way would be to access each item in turn by its name (or as `Item1`, `Item2`, etc., if you didn’t specify names), but C# provides another mechanism, called *deconstruction*. [Example 2-59](#) declares and initializes two tuples and then shows two different ways to deconstruct them.

Example 2-59. Constructing then deconstructing tuples

```
(int X, int Y) point1 = (40, 6);  
(int X, int Y) point2 = (12, 34);  
  
(int x, int y) = point1;  
Console.WriteLine($"1: {x}, {y}");  
(x, y) = point2;  
Console.WriteLine($"2: {x}, {y}");
```

Having defined `point1` and `point2`, this deconstructs `point1` into two variables, `x` and `y`. This particular form of deconstruction also declares the variables into which the tuple is being deconstructed. The alternative form is shown when we deconstruct `point2`—here, we’re deconstructing it into two variables that already exist, so there’s no need to declare them.

Until you become accustomed to this syntax, the first deconstruction example can seem confusingly similar to the first couple of lines, in which we declare and initialize new tuples. In those first couple of lines, the `(int X, int Y)` text signifies a tuple type with two `int` values named `X` and `Y`, but in the deconstruction line when we write `(int x, int y)`, we’re actually declaring two variables, each of type `int`. The only significant difference is that in the lines where we’re constructing new tuples, there’s a variable name before the `=` sign. (Also, we’re using uppercase names there, but that’s just a matter of convention. It would be entirely legal to write `(int x, int y) point3 = point1;`. That would declare a new tuple with two `int` values named `x` and `y`, stored in a variable named `point3`, initialized with the same values as are in `point1`. Equally, we could write `(int X, int Y) = point1;`. That would deconstruct `point` into two local variables called `X` and `Y`.)

You can mix the two forms of deconstruction: the lefthand side could be `(int z, x)`, which declares a new variable for the first part of the target, and uses an existing variable for the second. If you don’t need every element of a tuple, you can use an underscore, as [Example 2-60](#) shows. This is called a *discard*.

Example 2-60. Tuple deconstruction with discard

```
(_, int h) = point1;
```

The underscore character can appear in any number of places in the target, and it tells the compiler that we don't need that part of the tuple to be extracted into a variable.

Dynamic

C# defines a type called `dynamic`. This doesn't directly correspond to any CLR type—when we use `dynamic` in C#, the compiler presents it to the runtime as `object`, which is described in the next section. However, from the perspective of C# code, `dynamic` is a distinct type, and it enables some special behavior.

With `dynamic`, the compiler makes no attempt at compile time to check whether operations performed by code are likely to succeed. In other words, it effectively disables the statically typed behavior that we normally get with C#. You are free to attempt almost any operation on a `dynamic` variable—you can use arithmetic operators, you can attempt to invoke methods on it, you can try to assign it into variables of some other type, and you can try to get or set properties on it. When you do this, the compiler generates code that attempts to make sense of what you've asked it to do at runtime.

If you have come to C# from a language in which this sort of behavior is the norm (such as JavaScript), you might be tempted to use `dynamic` for everything because it works in a way you are used to. However, you should be aware that there are a couple of issues with it. First, it was designed with a particular scenario in mind: interoperability with certain pre-.NET Windows components. The Component Object Model (COM) in Windows is the basis for automatability of Microsoft 365 (perhaps still better known by its old name, Microsoft Office) and many other applications, and the scripting language built into Office is dynamic in nature. An upshot of this is that a lot of Office's automation APIs used to be hard

work to use from C#. One of the big drivers behind adding `dynamic` to the language was a desire to improve this.

As with all C# features, it was designed with broader applicability in mind and not simply as an Office interop feature. But since that was the most important scenario for this feature, you may find that its ability to support idioms you are familiar with from dynamic languages is disappointing. And the second issue to be aware of is that it is not an area of the language that is getting a lot of new work. When it was introduced, Microsoft went to considerable lengths to ensure that all dynamic behavior was as consistent as possible with the behavior you would have seen if the compiler had known at compile time what types you were going to be using.

This means that the infrastructure supporting `dynamic` (which is called the Dynamic Language Runtime, or DLR) has to replicate significant portions of C# behavior. However, the DLR has not been updated much since `dynamic` was added in C# 4.0 back in 2010, even though the language has seen many new features since then. Of course, `dynamic` still works, but its capabilities reflect how the language looked around a decade ago.

Even when it first appeared, `dynamic` had some limitations. There are some aspects of C# that depend on the availability of static type information, meaning that `dynamic` has always had some problems working with delegates and also with LINQ. So even from the start, it was at something of a disadvantage compared to using C# as intended, i.e., as a statically typed language.

Object

The last data type to get special recognition from the C# compiler is `object` (or `System.Object`, as the CLR calls it). This is the base class of almost⁸ all C# types. A variable of type `object` is able to refer to a value of any type that derives from `object`. This includes all numeric types, the `bool` and `string` types, and any custom types you can define using the keywords we'll look at in the next chapter, such as `class`, `record`, and `struct`. And it also includes all the types defined by the runtime libraries, with the exception of certain types that can only be stored on the stack and that are described in [Chapter 18](#).

So `object` is the ultimate general-purpose container. You can refer to almost anything with an `object` variable. We will return to this in [Chapter 6](#) when we look at inheritance.

Operators

Earlier you saw that expressions are sequences of operators and operands. I've shown some of the types that can be used as operands, so now it's time to see what operators C# offers. [Table 2-3](#) shows the ones that support common arithmetic operations.

Table 2-3. Basic arithmetic operators

Name	Example
Unary plus (does nothing)	<code>+x</code>
Negation (unary minus)	<code>-x</code>
Postincrement	<code>x++</code>
Postdecrement	<code>x--</code>
Preincrement	<code>++x</code>
Predecrement	<code>--x</code>
Addition	<code>x + y</code>
Subtraction	<code>x - y</code>
Multiplication	<code>x * y</code>
Division	<code>x / y</code>
Remainder	<code>x % y</code>

If you've had much experience with any other C-family language, all of these should seem familiar. If not, the most peculiar ones will probably be the increment and decrement operators. These have side effects: they add or subtract one from the variable to which they are applied (meaning they can be applied only to variables). With the postincrement and postdecrement, although the variable gets modified, the containing expres-

sion ends up getting the original value. So if `x` is a variable containing the value 5, the value of `x++` is also 5, even though the `x` variable will have a value of 6 after evaluating the `x++` expression. The pre- forms return the modified value, so if `x` is initially 5, `++x` produces the value 6, which is also the value of `x` after evaluating the expression.

Although the operators in [Table 2-3](#) are used in arithmetic, some are available on certain nonnumeric types. As you saw earlier, the `+` symbol represents concatenation when working with strings, and as you'll see in [Chapter 9](#), the addition and subtraction operators are also used for combining and removing delegates.

C# also offers some operators that perform certain binary operations on the bits that make up a value, shown in [Table 2-4](#). These are not available on floating-point types. (C# 11.0's generic math feature, described in [Chapter 4](#), does actually provide a way to use all but the shift operators on `double`, `float`, and `Half` values, but you still can't use these operators directly on variables of these types.)

Table 2-4. Binary integer operators

Name	Example
Bitwise negation	<code>~x</code>
Bitwise AND	<code>x & y</code>
Bitwise OR	<code>x y</code>
Bitwise XOR	<code>x ^ y</code>
Shift left	<code>x << y</code>
Shift right	<code>x >> y</code>
Unsigned shift right	<code>x >>> y</code>

The bitwise negation operator inverts all bits in an integer—any binary digit with a value of 1 becomes 0, and vice versa. The shift operators move all the bits left or right by the number of columns specified by the second operand. A left shift sets the least significant bits to 0. Right shifts of unsigned integers fill the higher order bits with 0. When you right shift

a signed integer, you can choose between `>>`, which fills the most significant bits with whatever value the leftmost bit already had (i.e., negative numbers remain negative because they keep their most significant bit set, while positive numbers keep their upper bit as 0, thus remaining positive), or `>>>`, which fills these bits with 0. The `>>>` operator is new in C# 11.0, and can be useful because some libraries use `int` to represent values that are really unsigned. (This happens sometimes because the CLS disallows the use of unsigned integer types, since not all languages support these.)

The bitwise AND, OR, and XOR (exclusive OR) operators perform Boolean logic operations on each bit of the two operands when applied to integers. These three operators are also available when the operands are of type `bool`. (In effect, these operators treat a `bool` as a one-digit binary number.) There are some additional operators available for `bool` values, shown in [Table 2-5](#). The `!` operator does to a `bool` what the `~` operator does to each bit in an integer.

Table 2-5. Operators for `bool`

Name	Example
Logical negation (also known as NOT)	<code>!x</code>
Conditional AND	<code>x && y</code>
Conditional OR	<code>x y</code>

If you have not used other C-family languages, the conditional versions of the AND and OR operators may be new to you. These evaluate their second operand only if necessary. For example, when evaluating `(a && b)`, if the expression `a` is `false`, the code generated by the compiler will not even attempt to evaluate `b`, because the result will be `false` no matter what value `b` has. Conversely, the conditional OR operator does not bother to evaluate its second operand if the first is `true`, because the result will be `true` regardless of the second operand's value. This is significant if the second operand's expression either contains elements that have side effects (such as method invocation) or might produce an error. For example, you often see code like that shown in [Example 2-61](#).

Example 2-61. The conditional AND operator

```
if (s is not null && s.Length > 10)
...
```

This checks to see if the variable `s` contains the special value `null`, meaning that it doesn't currently refer to any value. The use of the `&&` operator here is important, because if `s` is `null`, evaluating the expression `s.Length` would cause a runtime error. If we had used the `&` operator, the compiler would have generated code that always evaluates both operands, meaning that we would see a `NullReferenceException` at runtime if `s` is `null`. By using the conditional AND operator, we avoid that, because the second operand, `s.Length > 10`, will be evaluated only if `s` is not `null`.

Although code of the kind shown in [Example 2-61](#) was once common, it has gradually become much rarer thanks to a feature introduced back in C# 6.0, *null-conditional operators*. If you write `s?.Length` instead of just `s.Length`, the compiler generates code that checks `s` for `null` first, avoiding the `NullReferenceException`. This means the check can become just `if (s?.Length > 10)`. Furthermore, C#'s optional *nullable reference types* (a relatively new feature, discussed in [Chapter 3](#)) can help reduce the need for these kinds of tests for `null`.

[Example 2-61](#) tests to see if a property is greater than 10 by using the `>` operator. This is one of several *relational operators*, which allow us to compare values. They all take two operands and produce a `bool` result.

[Table 2-6](#) shows these, and they are supported for all numeric types.

Some operators are available on some other types too. For example, you can compare string values with the `==` and `!=` operators. (There is no built-in meaning for the other relational operators with `string` because different countries have different ideas about the order in which to sort strings. If you want ordered string comparison, .NET offers the `StringComparer` class, which requires you to select the rules by which you'd like your strings ordered.)

Table 2-6. Relational operators

Name	Example
Less than	<code>x < y</code>
Greater than	<code>x > y</code>
Less than or equal	<code>x <= y</code>
Greater than or equal	<code>x >= y</code>
Equal	<code>x == y</code>
Not equal	<code>x != y</code>

As is usual with C-family languages, the equality operator is a pair of equals signs. This is because a single equals sign means something else: it's an assignment, and assignments are expressions too. This can lead to an unfortunate problem: in some C-family languages, it's all too easy to write `if (x = y)` when you meant `if (x == y)`. Fortunately, this will usually produce a compiler error in C#, because C# has a special type to represent Boolean values. In languages that allow numbers to stand in for Booleans, both pieces of code are legal even if `x` and `y` are numbers. (The first means to assign the value of `y` into `x`, and then to execute the body of the `if` statement if that value is nonzero. That's very different than the second one, which doesn't change the value of anything and executes the body of the `if` statement only if `x` and `y` are equal.) But in C#, the first example would be meaningful only if `x` and `y` were both of type `bool`.⁹

Another feature that's common to the C family is the conditional operator. (Some people call it the ternary operator, because it's the only operator in the language that takes three operands.) It chooses between two expressions. More precisely, it evaluates its first operand, which must be a Boolean expression, and then returns the value of either the second or third operand, depending on whether the value of the first was `true` or `false`, respectively. [Example 2-62](#) uses this to pick the larger of two values. (This is just for illustration. In practice, you'd normally use .NET's `Math.Max` method, which has the same effect but is rather more readable. `Math.Max` also has the benefit that if you use expressions with side

effects, it will only evaluate each one once, something you can't do with the approach shown in [Example 2-62](#), because you end up writing each expression twice.)

Example 2-62. The conditional operator

```
int max = (x > y) ? x : y;
```

This illustrates why C and its successors have a reputation for terse syntax. If you are familiar with any language from this family, [Example 2-62](#) will be easy to read, but if you're not, its meaning might not be instantly clear. This will evaluate the expression before the `?` symbol, which is `(x > y)` in this case, and that's required to be an expression that produces a `bool`. (The parentheses are optional. I put them in to make the code easier to read.) If that is `true`, the expression between the `?` and `:` symbols is used (`x`, in this case); otherwise, the expression after the `:` symbol (`y` here) is used.

The conditional operator is similar to the conditional AND and OR operators in that it will evaluate only the operands it has to. It always evaluates its first operand, but it will never evaluate both the second and third operands. That means you can handle `null` values by writing something like [Example 2-63](#). This does not risk causing a `NullReferenceException`, because it will evaluate the third operand only if `s` is not `null`.

Example 2-63. Exploiting conditional evaluation

```
int characterCount = s is null ? 0 : s.Length;
```

However, in some cases, there are simpler ways of dealing with `null` values. Suppose you have a `string` variable, and if it's `null`, you'd like to use the empty string instead. You could write `(s is null ? "" : s)`. But you could just use the *null coalescing* operator instead, because it's designed for precisely this job. This operator, shown in [Example 2-64](#) (it's the `??` symbol), evaluates its first operand, and if that's non-null, that's the result of the expression. If the first operand is `null`, it evaluates its second operand and uses that instead.

Example 2-64. The null coalescing operator

```
string neverNull = s ?? "";
```

We could combine a null-conditional operator with the null coalescing operator to provide a more succinct alternative to [Example 2-63](#), shown in [Example 2-65](#).

Example 2-65. Null-conditional and null coalescing operators

```
int characterCount = s?.Length ?? 0;
```

One of the main benefits offered by the conditional, null-conditional, and null coalescing operators is that they often allow you to write a single expression in cases where you would otherwise have needed to write considerably more code. This can be particularly useful if you're using the expression as an argument to a method, as in [Example 2-66](#).

Example 2-66. Conditional expression as method argument

```
FadeVolume(gateOpen ? MaxVolume : 0.0, FadeDuration, FadeCurve.Linear);
```

Compare this with what you'd need to write if the conditional operator did not exist. You would need an `if` statement. (I'll get to `if` statements in the next section, but since this book is not for novices, I'm assuming you're familiar with the rough idea.) And you'd either need to introduce a local variable, as [Example 2-67](#) does, or you'd need to duplicate the method call in the two branches of the `if/else`, changing just the first argument. So, terse though the conditional and null coalescing operators are, they can remove a lot of clutter from your code.

Example 2-67. Life without the conditional operator

```
double targetVolume;  
if (gateOpen)  
{  
    targetVolume = MaxVolume;  
}
```

```
}  
else  
{  
    targetVolume = 0.0;  
}  
FadeVolume(targetVolume, FadeDuration, FadeCurve.Linear);
```

There is one last set of operators to look at: the *compound assignment* operators. These combine assignment with some other operation and are available for the `+`, `-`, `*`, `/`, `%`, `<<`, `>>`, `&`, `^`, `|`, and `??` operators. They enable you not to have to write the sort of code shown in [Example 2-68](#).

Example 2-68. Assignment and addition

```
x = x + 1;
```

We can write this assignment statement more compactly as the code in [Example 2-69](#). All the compound assignment operators take this form—you just stick an `=` on the end of the original operator.

Example 2-69. Compound assignment (addition)

```
x += 1;
```

This is a distinctive syntax that makes it very clear that we are modifying the value of a variable in some particular way. So, although those two snippets perform identical work, many developers find the second idiomatically preferable.

That's not quite a comprehensive list of operators. There are a few more specialized ones that I'll get to once we've looked at the areas of the language for which they were defined. (Some relate to classes and other types, some to inheritance, some to collections, and some to delegates. There are chapters coming up on all of these.) By the way, although I've been describing which operators are available on which types, it's possible to write a custom type that defines its own meanings for most of these. That's how .NET's `BigInteger` type can support the same arith-

metic operations as the built-in numeric types. I'll show how this can be done in [Chapter 3](#).

Flow Control

Most of the code we have examined so far executes statements in the order they are written and stops when it reaches the end. If that were the only possible way in which execution could flow through our code, C# would not be very useful. So, as you'd expect, it has a variety of constructs for writing loops and for deciding which code to execute based on inputs.

Boolean Decisions with if Statements

An `if` statement decides whether or not to run some particular statement depending on the value of a `bool` expression. For example, the `if` statement in [Example 2-70](#) will execute the block statement that shows a message only if the `age` variable's value is less than 18.

Example 2-70. Simple `if` statement

```
if (age < 18)
{
    Console.WriteLine("You are too young to buy alcohol in a bar in the UK.");
}
```

You don't have to use a block statement with an `if` statement. You can use any statement type as the body. A block is necessary only if you want the `if` statement to govern the execution of multiple statements.

However, some coding style guidelines recommend using a block in all cases. This is partly for consistency but also because it avoids a possible error when modifying the code at a later date: if you have a nonblock statement as the body of an `if`, and then you add another statement after that, intending it to be part of the same body, it can be easy to forget to add a block around the two statements, leading to code like that in [Example 2-71](#). The indentation suggests that the developer meant for the final statement to be part of the `if` statement's body, but C# ignores in-

dentation, so that final statement will always run. If you are in the habit of always using a block, you won't make this mistake.

Example 2-71. Probably not what was intended

```
if (authenticationCodesCorrect)
    SendTransferConfirmation();
    TransferFunds();
```

An `if` statement can optionally include an `else` part, which is followed by another statement that runs only if the `if` statement's expression evaluates to `false`. So [Example 2-72](#) will write either the first or the second message, depending on whether the `optimistic` variable is `true` or `false`.

Example 2-72. `if` and `else`

```
if (optimistic)
{
    Console.WriteLine("Glass half full");
}
else
{
    Console.WriteLine("Glass half empty");
}
```

The `else` keyword can be followed by any statement, and again, this is typically a block. However, there's one scenario in which most developers do not use a block for the body of the `else` part, and that's when they use another `if` statement. [Example 2-73](#) shows this—its first `if` statement has an `else` part, which has another `if` statement as its body.

Example 2-73. Picking one of several possibilities

```
if (temperatureInCelsius < 52)
{
    Console.WriteLine("Too cold");
}
else if (temperatureInCelsius > 58)
```

```

{
    Console.WriteLine("Too hot");
}
else
{
    Console.WriteLine("Just right");
}

```

This code still looks like it uses a block for that first `else`, but that block is actually the statement that forms the body of a second `if` statement. It's that second `if` statement that is the body of the `else`. If we were to stick rigidly to the rule of giving each `if` and `else` body its own block, we'd rewrite [Example 2-73](#) as [Example 2-74](#). This seems unnecessarily fussy, because the main risk that we're trying to avert by using blocks doesn't really apply in [Example 2-73](#).

Example 2-74. Overdoing the blocks

```

if (temperatureInCelsius < 52)
{
    Console.WriteLine("Too cold");
}
else
{
    if (temperatureInCelsius > 58)
    {
        Console.WriteLine("Too hot");
    }
    else
    {
        Console.WriteLine("Just right");
    }
}

```

Although we can chain `if` statements together as shown in [Example 2-73](#), C# offers a more specialized statement that can sometimes be easier to read.

Multiple Choice with switch Statements

A `switch` statement defines multiple groups of statements and either runs one group or does nothing at all, depending on the value of an input expression. As [Example 2-75](#) shows, you put the expression inside parentheses after the `switch` keyword, and after that, there's a region delimited by braces containing a series of `case` sections, defining the behavior for each anticipated value for the expression.

Example 2-75. A `switch` statement with strings

```
switch (workStatus)
{
    case "ManagerInRoom":
        WorkDiligently();
        break;

    case "HaveNonUrgentDeadline":
    case "HaveImminentDeadline":
        CheckSocialMedia();
        CheckEmail();
        CheckSocialMedia();
        ContemplateGettingOnWithSomeWork();
        CheckSocialMedia();
        CheckSocialMedia();
        break;

    case "DeadlineOvershot":
        WorkFuriously();
        break;

    default:
        CheckSocialMedia();
        CheckEmail();
        break;
}
```

As you can see, a single section can serve multiple possibilities—you can put several different `case` labels at the start of a section, and the statements in that section will run if any of those cases apply. You can also write a `default` section, which will run if none of the cases apply. A

`switch` statement does not have to be comprehensive, so if there is no `case` that matches the expression's value and there is no `default` section, the `switch` statement simply does nothing.

Unlike `if` statements, which take exactly one statement for the body, a `case` may be followed by multiple statements without needing to wrap them in a block. The sections in [Example 2-75](#) are delimited by `break` statements, which causes execution to jump to the end of the `switch` statement. This is not the only way to finish a section—strictly speaking, the rule imposed by the C# compiler is that the end point of the statement list for each `case` must not be reachable, so anything that causes execution to leave the `switch` statement is acceptable. You could use a `return` statement instead, or throw an exception, or you could even use a `goto` statement.

Some C-family languages (C, for example) allow *fall-through*, meaning that if execution is allowed to reach the end of the statements in a `case` section, it will continue with the next one. [Example 2-76](#) shows this style, and it is not allowed in C# because of the rule that requires the end of a `case` statement list not to be reachable.

Example 2-76. C-style fall-through, illegal in C#

```
switch (x)
{
    case "One":
        Console.WriteLine("One");
    case "Two": // This line will not compile
        Console.WriteLine("One or two");
        break;
}
```

C# outlaws this, because the vast majority of `case` sections do not fall through, and when they do in languages that allow it, it's often a mistake caused by the developer forgetting to write a `break` statement (or some other statement to break out of the `switch`). Accidental fall-through is likely to produce unwanted behavior, so C# requires more than the mere omission of a `break`: if you want fall-through, you must ask for it explic-

itly. As [Example 2-77](#) shows, we use the unloved `goto` keyword to express that we really do want one case to fall through into the next one.

Example 2-77. Fall-through in C#

```
switch (x)
{
    case "One":
        Console.WriteLine("One");
        goto case "Two";
    case "Two":
        Console.WriteLine("One or two");
        break;
}
```

This isn't technically a `goto` statement. It's a `goto case` statement and can be used only to jump within a `switch` block. C# also supports more general `goto` statements—you can add labels to your code and jump around within your methods. However, `goto` is heavily frowned upon, so the fall-through form offered by `goto case` statements seems to be the only use for this keyword that is considered respectable in modern society.

These examples have all used strings. You can also use `switch` with integer types, `char`, and any `enum` (a kind of type discussed in the next chapter). But `case` labels don't necessarily have to be constants: you can also use patterns, which are discussed later in this chapter.

Loops: while and do

C# supports the usual C-family loop mechanisms. [Example 2-78](#) shows a `while` loop. This takes a `bool` expression. It evaluates that expression, and if the result is `true`, it will execute the statement that follows. So far, this is just like an `if` statement, but the difference is that once the loop's embedded statement is complete, it then evaluates the expression again, and if it's `true` again, it will execute the embedded statement a second time. It will keep doing this until the expression evaluates to `false`. As

with `if` statements, the body of the loop does not need to be a block, but it usually is.

Example 2-78. A `while` loop

```
while (!reader.EndOfStream)
{
    Console.WriteLine(reader.ReadLine());
}
```

The body of the loop may decide to finish the loop early with a `break` statement. It does not matter whether the `while` expression is `true` or `false`—executing a `break` statement will always terminate the loop.

C# also offers the `continue` statement. Like a `break` statement, this terminates the current iteration, but unlike `break`, it will then reevaluate the `while` expression, so iteration may continue. Both `continue` and `break` jump straight to the end of the loop, but you could think of `continue` as jumping directly to the point just before the loop's closing `}`, while `break` jumps to the point just after. By the way, `continue` and `break` are also available for all of the other loop styles I'm about to show.

Because a `while` statement evaluates its expression before each iteration, it's possible for a `while` loop not to run its body at all. Sometimes, you may want to write a loop that runs at least once, only evaluating the `bool` expression after the first iteration. This is the purpose of a `do` loop, as shown in [Example 2-79](#).

Example 2-79. A `do` loop

```
char k;
do
{
    Console.WriteLine("Press x to exit");
    k = Console.ReadKey().KeyChar;
}
while (k != 'x');
```

Notice that [Example 2-79](#) ends in a semicolon, denoting the end of the statement. Compare this with the line containing the `while` keyword in [Example 2-78](#), which does not, despite otherwise looking very similar. That may look inconsistent, but it's not a typo. Putting a semicolon at the end of the line with the `while` keyword in [Example 2-78](#) would be legal, but it would change the meaning—it would indicate that we want the body of the `while` loop to be an empty statement. The block that followed would then be treated as a brand-new statement to execute after the loop completes. The code would get stuck in an infinite loop unless the reader were already at the end of the stream. (The compiler will issue a warning about a “Possible mistaken empty statement” if you do that, by the way.)

C-Style for Loops

Another style of loop that C# inherits from C is the `for` loop. This is similar to `while`, but it adds two features to that loop's `bool` expression: it provides a place to declare and/or initialize one or more variables that will remain in scope for as long as the loop runs, and it provides a place to perform some operation each time around the loop (in addition to the statement that forms the body of the loop). So the structure of a `for` loop looks like this:

```
for (initializer; condition; iterator) body
```

A very common application of this is to do something to all the elements in an array. [Example 2-80](#) shows a `for` loop that multiplies every element in an array by 2. The condition part works in exactly the same way as in a `while` loop—it determines whether the embedded statement forming the loop's body runs, and it will be evaluated before each iteration. Again, the body doesn't strictly have to be a block but usually is.

Example 2-80. Modifying array elements with a `for` loop

```
for (int i = 0; i < myArray.Length; i++)  
{
```

```
myArray[i] *= 2;  
}
```

The initializer in this example declares a variable called `i` and initializes it to 0. The initializer does not need to be a variable declaration—you can use any expression statement—but it’s a common choice. The initializer executes just once at the start of the loop, so this variable’s lifetime effectively begins just before the loop starts and ends when the loop finishes.

The iterator in [Example 2-80](#) just adds 1 to `i`. It runs at the end of each loop iteration, after the body runs and before the condition is reevaluated. (So if the condition is initially false, not only does the body not run, the iterator will never be evaluated.) C# does nothing with the result of the iterator expression—it is useful only for its side effects. So it doesn’t matter whether you write `i++`, `++i`, `i += 1`, or even `i = i + 1`.

A `for` loop doesn’t let you do anything that you couldn’t have achieved by writing a `while` loop and putting the initialization code before the loop and the iterator at the end of the loop body instead.¹⁰ However, there may be readability benefits. A `for` statement puts the code that defines how we loop in one place, separate from the code that defines what we do each time around the loop, which might help those reading the code to understand what it does. They don’t have to scan down to the end of a long loop to find the iterator statement (although a long loop body that trails over pages of code is generally considered to be bad practice, so this last benefit is a little dubious).

Both the initializer and the iterator can contain lists, as [Example 2-81](#) shows, although in this particular case it isn’t terribly useful—since all the iterators run every time around, `i` and `j` will have the same value as each other throughout.

Example 2-81. Multiple initializers and iterators

```
for (int i = 0, j = 0; i < myArray.Length; i++, j++)  
...
```

You can't write a single `for` loop that performs a multidimensional iteration. If you want that, you would nest one loop inside another, as [Example 2-82](#) illustrates.

Example 2-82. Nested `for` loops

```
for (int j = 0; j < height; ++j)
{
    for (int i = 0; i < width; ++i)
    {
        ...
    }
}
```

Although [Example 2-80](#) shows a common enough idiom for iterating through arrays, you will often use a different, more specialized construct.

Collection Iteration with `foreach` Loops

C# offers a style of loop that is not universal in C-family languages. The `foreach` loop is designed for iterating through collections. A `foreach` loop fits this pattern:

```
foreach (item-type iteration-variable in collection) body
```

The `collection` is an expression whose type must match a particular pattern recognized by the compiler. The runtime libraries' `IEnumerable<T>` interface, which we'll be looking at in [Chapter 5](#), matches this pattern, although the compiler doesn't actually require an implementation of that interface—it just requires the collection to have a `GetEnumerator` method that resembles the one defined by that interface. [Example 2-83](#) uses `foreach` to show all the strings in an array. (All arrays provide the method that `foreach` requires.)

Example 2-83. Iterating over a collection with `foreach`

```
string[] messages = GetMessagesFromSomewhere();
foreach (string message in messages)
```

```
{  
    Console.WriteLine(message);  
}
```

This loop will run the body once for each item in the array. The *iteration variable* (`message`, in this example) is different each time around the loop and will refer to the item for the current iteration.

In one way, this is less flexible than the `for`-based loop shown in [Example 2-80](#): a `foreach` loop cannot modify the collection it iterates over. That's because not all collections support modification.

`IEnumerable<T>` demands very little of its collections—it does not require modifiability, random access, or even the ability to know up front how many items the collection provides. (In fact, `IEnumerable<T>` is able to support never-ending collections. For example, it is perfectly legal to write an implementation that will return random numbers for as long as you care to keep fetching values.)

But `foreach` offers two advantages over `for`. One advantage is subjective and therefore debatable: it's a bit more readable. But significantly, it's also more general. If you're writing methods that do things to collections, those methods will be more broadly applicable if they use `foreach` rather than `for`, because you'll be able to accept an `IEnumerable<T>`.

[Example 2-84](#) can work with any collection that contains strings, rather than being limited to arrays.

Example 2-84. General-purpose collection iteration

```
public static void ShowMessages(IEnumerable<string> messages)  
{  
    foreach (string message in messages)  
    {  
        Console.WriteLine(message);  
    }  
}
```

This code can work with collection types that do not support random access, such as the `LinkedList<T>` class described in [Chapter 5](#). It can also process lazy collections that decide what items to produce on demand, in-

cluding those produced by iterator functions, also shown in [Chapter 5](#), and by certain LINQ queries, as described in [Chapter 10](#).

Patterns

There's one last essential mechanism to look at in C#: *patterns*. A pattern describes one or more criteria that a value can be tested against. You've already seen some simple patterns in action: each `case` in a `switch` specifies a pattern. But as we'll now see, there are many kinds of patterns, and they aren't just for `switch` statements.

The `switch` examples earlier, such as [Example 2-75](#), used one of the simplest pattern types: they were all *constant patterns*. With these, you specify just a constant value, and an expression matches this pattern if it has that value. [Example 2-85](#) shows a more interesting kind of pattern: it uses *declaration patterns*. An expression matches a declaration pattern if it has the specified type. As you saw earlier in ["Object"](#), some variables are capable of holding a variety of different types. Variables of type `object` are an extreme case of this, since they can hold more or less anything.

Language features such as *interfaces* (discussed in [Chapter 3](#)), generics ([Chapter 4](#)), and inheritance ([Chapter 6](#)) can lead to scenarios where the static type of a variable provides more information than the anything-goes `object` type but still leaves latitude for a range of possible types at runtime. Declaration patterns can be useful in these cases.

Example 2-85. Declaration patterns

```
switch (o)
{
    case string s:
        Console.WriteLine($"A piece of string is {s.Length} long");
        break;

    case int i:
        Console.WriteLine($"That's numberwang! {i}");
        break;
}
```

Declaration patterns have an interesting characteristic that constant ones do not: as well as the Boolean match/no-match common to all patterns, a declaration pattern produces an additional output. Each `case` in [Example 2-85](#) introduces a variable, which the code for that `case` then goes on to use. This output is just the input but copied into a variable with the specified static type. So that first `case` will match if `o` turns out to be a `string`, in which case we can access it through the `s` variable (which is why that `s.Length` expression compiles correctly; `o.Length` would not if `o` is of type `object`).

Sometimes, you won't actually need a declaration pattern's output—it might be enough just to know that the input matched a pattern. One way to handle these cases is with a *discard*: if you put an underscore (`_`) in the place where the output variable name would normally go, that tells the C# compiler that you are only interested in whether the value matches the type. There's a more succinct alternative: *type patterns*. A type pattern looks and works like a declaration pattern without the variable—as [Example 2-86](#) shows, the pattern consists of just the type name.

Example 2-86. Type patterns

```
switch (o)
{
    case string:
        Console.WriteLine("This is a piece of string");
        break;

    case int:
        Console.WriteLine("That's numberwang!");
        break;
}
```

Some patterns do a little more work to produce their output. For example, [Example 2-87](#) shows a *positional pattern* that matches any tuple containing a pair of `int` values and extracts those values into two variables, `x` and `y`.

Example 2-87. Positional pattern

```
case (int x, int y):  
    Console.WriteLine($"I know where it's at: {x}, {y}");  
    break;
```

Positional patterns are an example of a *recursive pattern*: they are patterns that contain patterns. In this case, this positional pattern contains a declaration pattern as each of its children. But as [Example 2-88](#) shows, we can use constant values in each position to match tuples with specific values.

Example 2-88. Positional patterns with constant values

```
switch (p)  
{  
    case (0, 0):  
        Console.WriteLine("How original");  
        break;  
  
    case (0, 1):  
    case (1, 0):  
        Console.WriteLine("What an absolute unit");  
        break;  
  
    case (1, 1):  
        Console.WriteLine("Be there and be square");  
        break;  
}
```

We can mix things up, because positional patterns can contain different pattern types in each position. [Example 2-89](#) shows a positional pattern with a constant pattern in the first position and a declaration pattern in the second.

Example 2-89. Positional pattern with constant and declaration patterns

```
case (0, int y):  
    Console.WriteLine($"This is on the X axis at height {y}");
```

```
break;
```

If you are a fan of `var`, you might be wondering if you can write something like [Example 2-90](#). This will work, and the static types of the `x` and `y` variables here will depend on the type of the pattern's input expression. If the compiler can determine how the expression deconstructs (for example, if the `switch` statement input's static type is an `(int, int)` tuple), then it will use this information to determine the output variables' static types. In cases where this is unknown, but it's still conceivable that this pattern could match (for example, if the input is `object`), then `x` and `y` here will also have type `object`.

Example 2-90. Positional pattern with `var`

```
case (var x, var y):  
    Console.WriteLine($"I know where it's at: {x}, {y}");  
    break;
```

NOTE

The compiler will reject patterns in cases where it can determine that a match is impossible. For example, if it knows the input type is a `(string, int, bool)` tuple, it cannot possibly match a positional pattern with only two child patterns, so C# won't let you try.

[Example 2-90](#) shows an unusual case where using `var` instead of an explicit type can introduce a significant change of behavior. These *`var` patterns* differ in one important respect from the declaration patterns in [Example 2-87](#): a `var` pattern always matches its input, whereas a declaration pattern inspects its input's type to determine at runtime whether it matches. This check might be optimized away in practice—there are cases where a declaration pattern will always match because its input type is known at compile time. But the only way to express in your code that you definitely don't want the child patterns in a positional pattern to perform a runtime check is to use `var`. So although a positional pattern containing declaration patterns strongly resembles the deconstruction syntax shown in [Example 2-59](#), the behavior is quite different. [Example 2-87](#) is in

effect asking three questions at runtime: Is the value a 2-tuple, is the first value an `int`, is the second value an `int`? (So it would work for tuples with a static type of `(object, object)`, as long as each value is an `int` at runtime.) This shouldn't really be surprising: the point of patterns is to test at runtime whether a value has certain characteristics. However, with some recursive patterns, you may find yourself wanting to express a mixture of runtime matching (for example, is this thing a `string`?) combined with statically typed deconstruction (for example, if this is a `string`, I'd like to extract its `Length` property, which I believe to be of type `int`, and I want a compiler error if that belief turns out to be wrong). Patterns are not designed to do this, so it's best not to try to use them that way.

What if we don't need to use all of the items in the tuple? You already know one way to handle that. Since we can use any pattern in each position, we could use a declaration pattern that discards its result in, say, the second position: `(int x, int _)`. Or we could use a type pattern: `(int x, int)`. However, [Example 2-91](#) shows a shorter alternative: instead of a type pattern, we can use just a lone underscore. This is a *discard pattern*. You can use it in a recursive pattern anyplace a pattern is required but where you want to indicate that anything will do in that particular position and that you don't need to know what it was.

Example 2-91. Positional pattern with discard pattern

```
case (int x, _):  
    Console.WriteLine($"At X: {x}. As for Y, who knows?");  
    break;
```

This has subtly different semantics than the discarding declaration pattern or the type pattern: those patterns will check at runtime that the value to be discarded has the specified type, and the pattern will only match if this check succeeds. But a discard pattern always matches, so this would match `(10, 20)`, `(10, "Foo")`, and `(10, (20, 30))`, for example.

You might be looking at these tuple patterns and wondering if we can do something similar with lists. C# 11.0 introduced list patterns, enabling us

to write the code in [Example 2-92](#).

Example 2-92. List patterns

```
case [int x]:
    Console.WriteLine($"Scalar: {x}");
    break;
case [int x, int y]:
    Console.WriteLine($"2D vector: ({x}, {y})");
    break;
case [int x, int y, int z]:
    Console.WriteLine($"3D vector: ({x}, {y}, {z})");
    break;
```

Each of these patterns starts by asking the question: Is this a list? As you'd expect, arrays qualify, as does the runtime library's `List<T>`, or indeed anything implementing `ICollection<T>`. List patterns don't demand any particular type though. They consider something to be a list if it offers either a `Length` or `Count` property, and a numeric indexer. (Indexers are described in ["Indexers"](#), but in practice, any type that can be used with syntax such as `items[42]` qualifies.)

NOTE

Most patterns perform all their tests at runtime, but list patterns are an exception: the compiler checks that the source type is a list at compile time. This is because it needs to generate different code for different kinds of lists—although `List<T>` and arrays both support numeric indexing, the compiler emits quite different IL for each.

If the input is a list, the patterns in [Example 2-92](#) will then ask: Is it the right length? (This test *does* happen at runtime.) Suppose the input is an array with two elements. Only the middle pattern will match in that case. Finally, it will test that the two elements match the two patterns supplied—like positional patterns, list patterns are recursive. These examples use declaration patterns, but you can put any pattern you like in each position of a list pattern (including another list pattern, if you are looking for a list of lists).

List patterns don't have to require a fixed size. The `..` in [Example 2-93](#) is a *slice pattern*, and it indicates that there can be any number of additional elements at that point. So inputs of `["alpha", "papa", "omega"]` or `["alpha", 42, (1.2, true), null, "omega"]` would match. Also, `["alpha", "omega"]` would match, because a slice will accept zero additional elements. (Slice patterns can appear only inside a list pattern, so these are also new in C# 11.0.)

Example 2-93. A list pattern containing a slice pattern

```
if (list is ["alpha", .., "omega"])
{
    Console.WriteLine("That seems to be in order");
}
```

[Example 2-93](#) is the simplest form of slice pattern: it essentially says we don't care what, if anything, appears at that point in the list. But what if we wanted to do something with that part? The `..` can optionally be followed by any other pattern, which will be applied to whichever part of the list the slice represents. [Example 2-94](#) shows a slice pattern that uses a declaration pattern.

Example 2-94. A list pattern containing a slice pattern with a nested pattern

```
if (list is ["alpha", .. string[] theRest, "omega"])
{
    Console.WriteLine(string.Join(", ", theRest));
}
```

This causes an extra compile-time test—the compiler will reject patterns of this kind if it can't determine how to extract the middle section of the list, or if it can, but the resulting type is wrong. (It effectively uses the range syntax, so `theRest` will be extracted with code equivalent to `list[1..^1]`. [“Addressing Elements with Index and Range Syntax”](#) describes this syntax, and not all lists support it.) So [Example 2-94](#) would compile if `list` was of type `string[]`, but not if it was an `IList<T>`, because the `IList<T>` interface doesn't require all implementations to sup-

port range indexing. It would also fail if `list` was a type that did support range indexing but which returned the wrong type. For example, if `list` were a `List<string>` (which implements `IList<T>` but also provides a range indexer—see [Chapter 3](#) for details), then a range indexer would be available, but it returns another `List<string>`, and since that is incompatible with `string[]`, the compiler would reject [Example 2-94](#). (The declaration pattern for the slice would need to specify `List<string>`, or some compatible type such as `IEnumerable<string>`, instead of `string[]`.)

A slice pattern can use any kind of pattern—it doesn't have to be a declaration pattern. So in addition to checking the list length and all the other nested patterns (checking whether `list` starts with "alpha" and ends with "omega" in this case) it will also perform whatever runtime tests the slice pattern's nested pattern requires. The whole list pattern only succeeds if all these steps succeed. And in this case, since the nested pattern is a declaration pattern, it will also initialize `theRest` with an array containing everything except the first and last elements of `list`.

NOTE

A list pattern may contain at most one slice. This is because multiple slices would significantly increase the complexity of pattern matching. To match `[1, .., 2]` requires a length check (`>=2`) and to inspect the first and last elements. But to match `[1, .., 2, 3, .. 4]` would involve searching the entire list to see if the values 2 and 3 occur consecutively at any point. That would be a powerful capability, but it would make it easy to write very expensive or ambiguous patterns by mistake.

Positional and list patterns are not the only recursive ones: you can also write a *property pattern*. We'll look at properties in detail in the next chapter, but for now it's enough to know that they are members of a type that provide some sort of information, such as the `string` type's `Length` property, which returns an `int` telling you how many code units the string contains. [Example 2-95](#) shows a property pattern that inspects this `Length` property.

Example 2-95. Property pattern

```
case string { Length: 0 }:  
    Console.WriteLine("How long is a piece of string? Not very!");  
    break;
```

This property pattern starts with a type name, so it effectively incorporates the behavior of a type pattern in addition to its property-based tests. (You can omit this in cases where the type of the pattern's input is sufficiently specific to identify the property. For example, if the input in this case already had a static of type `string`, we could omit this.) This is then followed by a section in braces listing each of the properties that the pattern wants to inspect and the pattern to apply for that property. (These child patterns are what make this another recursive pattern.) So this example first checks to see if the input is a `string`. If it is, it then applies a constant pattern to the string's `Length`, so this pattern matches only if the input is a `string` with `Length` of 0.

Property patterns can optionally specify an output. [Example 2-95](#) doesn't do this. [Example 2-96](#) shows the syntax, although in this particular case it's not terribly useful because this pattern will ensure that `s` only ever refers to an empty string.

Example 2-96. Property pattern with output

```
case string { Length: 0 } s:  
    Console.WriteLine($"How long is a piece of string? This long: {s.Length}");  
    break;
```

Since each property in a property pattern contains a nested pattern, those too can produce outputs, as [Example 2-97](#) shows.

Example 2-97. Property pattern with nested pattern with output

```
case string { Length: int length }:  
    Console.WriteLine($"How long is a piece of string? This long: {length}");  
    break;
```

You can nest property patterns within property patterns. [Example 2-98](#) uses this to inspect the operating system version reported by `Environment.OSVersion`, testing whether the major version is equal to 10.

Example 2-98. Property pattern with nested property pattern

```
switch (Environment.OSVersion)
{
    case { Version: { Major: 10 } }:
        Console.WriteLine("Windows 10, 11, or later");
        break;
}
```

There is a more succinct way to express this. You can replace the `case` in [Example 2-98](#) with [Example 2-99](#). It has exactly the same effect but is a more compact, and arguably more readable, expression of the intent.

Example 2-99. Extended property pattern

```
case { Version.Major: 10 }:
    Console.WriteLine("Windows 10, 11, or later");
    break;
```

Combining and Negating Patterns

C# offers three logical operations for use in patterns: `and`, `or`, and `not`. The simplest of these is `not`, and it lets you invert the meaning of a pattern. [Example 2-100](#) uses this to ensure it runs certain code only if a variable is non-null. This applies negation (`not`) to a constant pattern: the `null` here is interpreted as a constant pattern. If we had written just `null`, the pattern would match when the value is null, but with `not null` the pattern matches when it is not.

Example 2-100. Detecting non-nullness with pattern negation

```
case not null:
    Console.WriteLine($"User's middle name is: {middleName}");
```

```
break;
```

We can use `and` and `or` to combine pairs of patterns. (These are officially called *conjunctive* and *disjunctive* patterns; apparently the C# language designers are fans of formal propositional logic.) If we combine two patterns with `and`, the result is a pattern that matches only if both of the constituent patterns match. For example, if you wanted to write code that had something against my middle name, you could use the approach shown in [Example 2-101](#). This also shows that you can use a mixture of these logical operations: this uses both `and` and `not`.

Example 2-101. Using pattern conjunction (`and`) and negation (`not`)

```
case not null and not "David":  
    Console.WriteLine($"User's middle name is: {middleName}");  
    break;
```

We can use `or` in a similar way, and the effect is a pattern that matches its input if either of its constituent patterns matches. You can build up larger combinations through repeated use of `and` and/or `or`.

Relational Patterns

Patterns can use the `<`, `<=`, `>=`, and `>` operators when the pattern's type supports these kinds of comparison. [Example 2-102](#) shows a `switch` statement that includes two *relational patterns*, as patterns based on these operators are called.

Example 2-102. Relational patterns

```
switch (value)  
{  
    case > 0: Console.WriteLine("Positive"); break;  
    case < 0: Console.WriteLine("Negative"); break;  
    default: Console.WriteLine("Neither strictly positive nor negative"); break;  
};
```

You can use relational patterns in any place that any other pattern can be used. So they could appear inside a positional pattern (e.g., if you wanted to match points on the Y axis, above the X axis you could write `(0, > 0)`). [Example 2-103](#) uses two relational patterns as the constituents of a conjunction to express the requirement that a value falls within a particular range.

Example 2-103. Using relational patterns in a conjunction

```
case >= 168 and <= 189:  
    Console.WriteLine("Is within inner 90 percentiles");  
    break;
```

Relational patterns support comparisons only with constants. You cannot replace the numbers in the preceding examples with variables.

Getting More Specific with when

Sometimes, the built-in pattern types won't provide the level of precision you need. For example, with positional patterns, we've seen how to write patterns that match, say, any pair of values, or any pair of numbers, or a pair of numbers where one has a particular value. But what if you want to match a pair of numbers where the first is higher than the second?

This isn't a big conceptual leap, but there's no built-in support for this—relational patterns can't do this because they can compare only with constants. We could detect the condition with an `if` statement of course, but it would seem a shame to have to restructure our code from a `switch` to a series of `if` and `else` statements just to make this small step forward. Fortunately we don't have to.

Any pattern in a `case` label can be qualified by adding a `when` clause. It allows a Boolean expression to be included. This will be evaluated if the value matches the main part of the pattern, and the value will match the pattern as a whole only if the `when` clause is true. [Example 2-104](#) shows a positional pattern with a `when` clause that matches pairs of numbers in which the first number is larger than the second.

Example 2-104. Pattern with `when` clause

```
case (int w, int h) when w > h:
    Console.WriteLine("Landscape");
break;
```

Patterns in Expressions

All of the patterns I've shown so far appear in `case` labels as part of a `switch` statement. This is not the only way to use patterns. They can also appear inside expressions. To see how this can be useful, look first at the `switch` statement in [Example 2-105](#). The intent here is to return a single value determined by the input, but it's a little clumsy: I have had to write four separate `return` statements to express that.

Example 2-105. Patterns, but not in expressions

```
switch (shape)
{
    case (int w, int h) when w < h: return "Portrait";
    case (int w, int h) when w > h: return "Landscape";
    case (int _, int _): return "Square";
    default: return "Unknown";
}
```

[Example 2-106](#) shows code that performs the same job but rewritten to use a *switch expression*. As with a `switch` statement, a `switch` expression contains a list of patterns. The difference is that whereas labels in a `switch` statement are followed by a list of statements, in a `switch` expression each pattern is followed by a single expression. The value of a `switch` expression is the result of evaluating the expression associated with the first pattern that matches.

Example 2-106. A `switch` expression

```
return shape switch
{
    (int w, int h) when w < h => "Portrait",
```

```
(int w, int h) when w > h => "Landscape",  
(int _, int _) => "Square",  
_ => "Unknown"  
};
```

`switch` expressions look quite different than `switch` statements, because they don't use the `case` keyword. Instead, they just dive straight in with the pattern, and then use `=>` between the pattern and its corresponding expression. There are a few reasons for this. First, it makes `switch` expressions a bit more compact. Expressions are generally used inside other things—in this case, the `switch` expression is the value of a `return` statement, but you might also use these as a method argument or anywhere else an expression is allowed—so we generally want them to be succinct. Secondly, using `case` here could have led to confusion because the rules for what follows each `case` would be different for `switch` statements and `switch` expressions: in a `switch` statement, each `case` label is followed by one or more statements, but in a `switch` expression, each pattern needs to be followed by a single expression. Finally, although `switch` expressions were added to C# fairly recently, this sort of construct has been around in other languages for many years. C#'s version of it more closely resembles equivalents from other languages than it would have done if the expression form used the `case` keyword.

Notice that the final pattern in [Example 2-106](#) is a discard pattern. This will match anything, and it's there to ensure that the pattern is exhaustive, i.e., that it covers all possible cases. (It has a similar effect to a `default` section in a `switch` statement.) Unlike a `switch` statement, where it's OK for there to be no matches, a `switch` expression has to produce a result, so the compiler will warn you if your patterns don't handle all possible cases for the input type. It would complain in this situation if we were to remove that final case, assuming the `shape` input is of type `object`. (Conversely, if `shape` were of type `(int, int)`, we would have to remove that final case, because the first three cases in fact cover all possible values for that type and the compiler will produce an error telling us that the final pattern will never apply.) If you ignore this warning, and then at runtime you evaluate a `switch` expression with an un-

matchable value, it will throw a `SwitchExpressionException`. Exceptions are described in [Chapter 8](#).

There's one more way to use a pattern in an expression, and that's with the `is` keyword. It turns any pattern into a Boolean expression.

[Example 2-107](#) shows a simple example that determines whether a value is a tuple containing two integers.

Example 2-107. An `is` expression

```
bool isPoint = value is (int, int);
```

This also provides a way to ensure that a value is non-null before proceeding. [Example 2-108](#) combines a negation with a constant pattern testing for `null`.

Example 2-108. Testing for non-nullness with `is`

```
if (s is not null)
{
    Console.WriteLine(s.Length);
}
```

You might be wondering why we wouldn't just write `s != null`. In most cases that will work, but it has a potential problem: types are free to customize the behavior of comparison operators such as `!=`. The advantage of the approach in [Example 2-108](#) is that it will invariably perform just a simple comparison with `null` even with types that have customized the behavior of `!=` and `==`. (The positive form, `is null`, has the same advantage.)

As with patterns in `switch` statements or expressions, the pattern in an `is` expression can extract values from its source. Like [Example 2-107](#), the pattern in [Example 2-109](#) tests whether a value is a tuple containing two integers but goes on to use the two values from the tuple.

Example 2-109. Using the values from an `is` expression's pattern

```
if (value is (int x, int y))
{
    Console.WriteLine($"X: {x}, Y: {y}");
}
```

New variables introduced in this way by an `is` expression remain in scope after their containing statement. So in both these examples, `x` and `y` would continue to be in scope until the end of the containing block. Since the pattern in [Example 2-109](#) is in the `if` statement's condition expression, that means these variables remain in scope after the body block. However, if you try to use them outside of the body, you'll find that the compiler's definite assignment rules will tell you that they are uninitialized. It allows [Example 2-109](#) because it knows that the body of the `if` statement will run only if the pattern matches, so in that case `x` and `y` will have been initialized and are safe to use.

Patterns in `is` expressions cannot include a `when` clause. It would be redundant: the result is a Boolean expression, so you can just add on any qualification you require using the normal Boolean operators, as [Example 2-110](#) shows.

Example 2-110. No need for `when` in an `is` expression's pattern

```
if (value is (int w, int h) && w < h)
{
    Console.WriteLine($"(Portrait) Width: {w}, Height: {h}");
}
```

Summary

In this chapter, I showed the nuts and bolts of C# code—variables, statements, expressions, basic data types, operators, flow control, and patterns. Now it's time to take a look at the broader structure of a program. All code in C# programs must belong to a type, and types are the topic of the next chapter.

- ¹ See Alan Turing's seminal work on computation for details. Charles Petzold's *The Annotated Turing* (John Wiley & Sons) is an excellent guide to the relevant paper.
- ² If you're new to C-family languages, the `+=` operator may be unfamiliar. It is a *compound assignment* operator, described later in this chapter. I'm using it here to increase `errorCount` by one.
- ³ In the absence of parentheses, C# has rules of *precedence* that determine the order in which operators are evaluated. For the full (and not very interesting) details, consult the documentation. In this example, because division has higher precedence than addition, without parentheses the expression would evaluate to 14.
- ⁴ This is guaranteed only for correctly aligned 32-bit types. However, C# aligns them correctly by default, and you'd normally encounter misaligned data only if your code needs to call out into unmanaged code.
- ⁵ A decimal, therefore, doesn't use all of its 128 bits. Making it smaller would cause alignment difficulties, and using the additional bits for extra precision would have a significant performance impact, because on most CPUs, integers whose length is a multiple of 32 bits are easier to deal with than the alternatives.
- ⁶ Promotions are not in fact a feature of C#. There is a more general mechanism: conversion operators. C# defines intrinsic implicit conversion operators for the built-in data types. The promotions discussed here occur as a result of the compiler following its usual rules for conversions.
- ⁷ A *property* is a member of a type that represents a value that can be read or modified or both. [Chapter 3](#) describes properties in detail.
- ⁸ There are some specialized exceptions, such as pointer types.
- ⁹ Language pedants will note that it will also be meaningful in certain situations where custom implicit conversions to `bool` are available. We'll get to custom conversions in [Chapter 3](#).
- ¹⁰ A `continue` statement complicates matters, because it provides a way to move to the next iteration without getting all the way to the end of the loop body. Even so, you could still reproduce the effect of the iterator when using `continue` statements—it would just require more work.

